Name - Gauri Raut Section - C(C4) Roll No - 52 ML lab - 4

In [ ]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib as plt
```

In [ ]: In
```
df=pd.read_csv('winequality-red.csv')
```

[ ]: Out[
```
df.head()
```

]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alco
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.5	
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	6	
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.6	
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	8	
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.6	

In [ ]:
```
df.tail()
```

Out[ ]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates a
1594	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.5
1595	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	8
1596	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.7
1597	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	6
1598	6.0	0.310	0.47	3.6	0.067	18.0	42.0	0.99549	3.39	0.7

In [ ]:
```
df.shape
```

Out[ ]:
```
(1599, 12)
```

In [ ]:
```
df.describe()
```

Out[ ]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	freesulfur	total su dio
count	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000
mean	8.319637	0.527821	0.270976	2.538806	0.087467	15.874922	46.467
std	1.741096	0.179060	0.194801	1.409928	0.047065	10.460157	32.895
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000
25%	7.100000	0.390000	0.090000	1.900000	0.070000	7.000000	22.000
50%	7.900000	0.520000	0.260000	2.200000	0.079000	14.000000	38.000
75%	9.200000	0.640000	0.420000	2.600000	0.090000	21.000000	62.000
max	15.900000	1.580000	1.000000	15.500000	0.611000	72.000000	289.000

In [ ]:

```
df.info
```

Out[ ]:

<bound method DataFrame.info of fixed acidity volatile acidity c
itric acid 0 residual sugar chlorides \
0.076 7.4 0.700 0.00 1.9
1
0.098 7.8 0.880 0.00 2.6
2
0.092 7.8 0.760 0.04 2.3
3
0.075 11.2 0.280 0.56 1.9
4
0.076 7.4 0.700 0.00 1.9
...
... ...
1594
0.090 6.2 0.600 0.08 2.0
1595
0.062 5.9 0.550 0.10 2.2
1596
0.076 6.3 0.510 0.13 2.3
1597
0.075 5.9 0.645 0.12 2.0
1598
0.067 6.0 0.310 0.47 3.6

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphate
s \					
0	11.0	34.0	0.99780	3.51	0.5
6					
1	25.0	67.0	0.99680	3.20	0.6
8					
2	15.0	54.0	0.99700	3.26	0.6
5					
3	17.0	60.0	0.99800	3.16	0.5
8					
4	11.0	34.0	0.99780	3.51	0.5
6					
... ...	...	...	...	...	
1594					
8	32.0	44.0	0.99490	3.45	0.5
1595					
6	39.0	51.0	0.99512	3.52	0.7
1596					
5	29.0	40.0	0.99574	3.42	0.7
1597					
1	32.0	44.0	0.99547	3.57	0.7
1598					
6	18.0	42.0	0.99549	3.39	0.6

	alcohol	quality
0	9.4	5
1	9.8	5
2	9.8	5
3	9.8	6

```
4     ...          9.4          5
1594                ...        ...
1595           10.5          5
1596           11.2          6
1597           11.0          6
1598           10.2          5
               11.0          6

[1599 rows x 12 columns]>
```

In [ ]:
```
df.isnull().sum()
```

Out[ ]:
```
fixed     acidity     volatile    0
acidity citric acid               0
residual sugar chlorides          0
free  sulfur  dioxide  total      0
sulfur dioxide density            0
pH                                0
sulphates                         0
alcohol                           0
quality                           0
dtype: int64                      0
                                  0
                                  0
```

In [ ]:
```
df.duplicated().sum()
```

Out[ ]:
```
np.int64(240)
```

In [ ]: In [ ]:
```
df=df.drop_duplicates()
```

Out[ ]:
```
df.duplicated().sum()
```

```
np.int64(0)
```

In [ ]:

Out[ ]:
```
df.columns
```

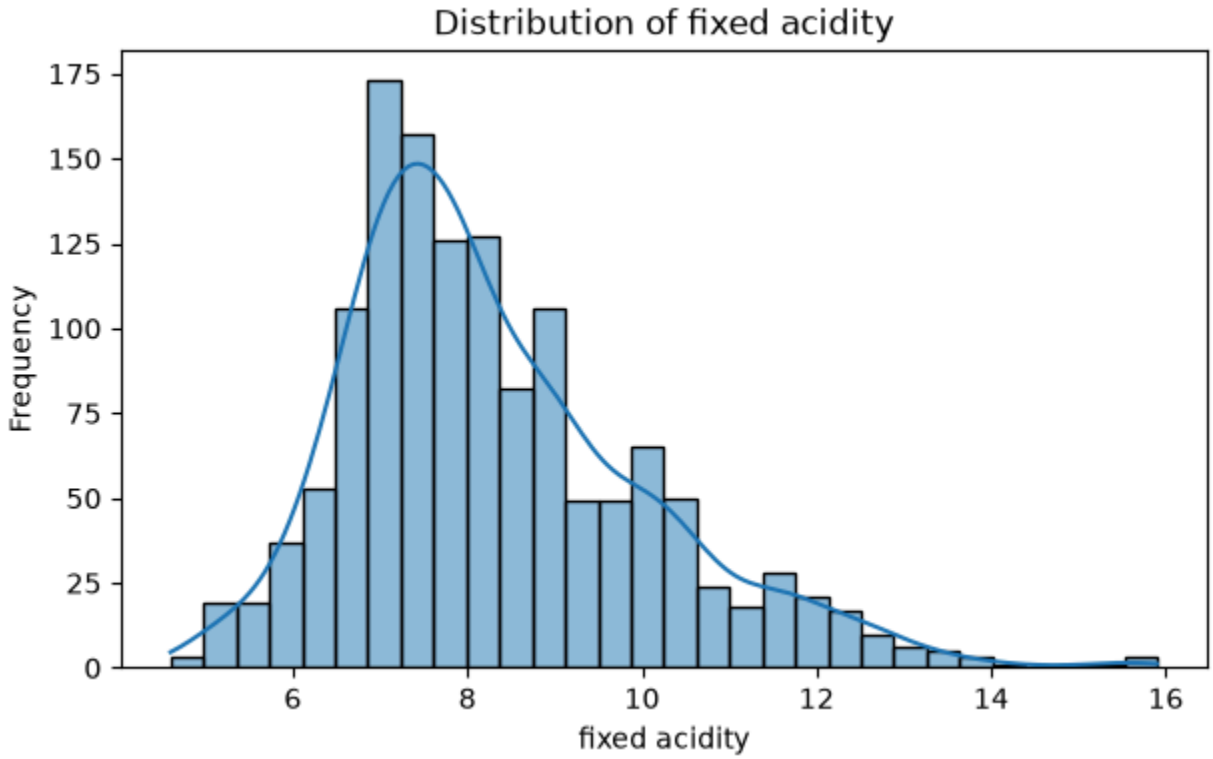
```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sug
ar',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'dens
ity',
       'pH', 'sulphates', 'alcohol', 'quality'],
      dtype='str')
```

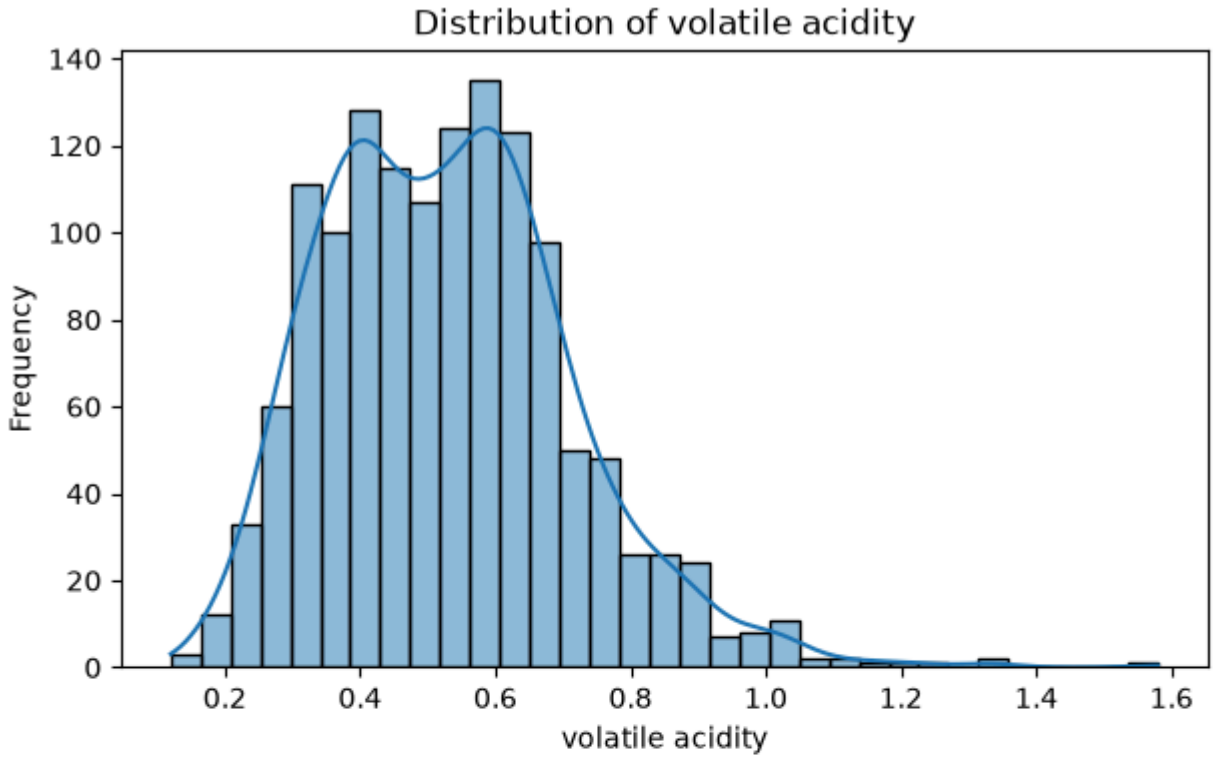
UNIVARIATE ANALYSIS

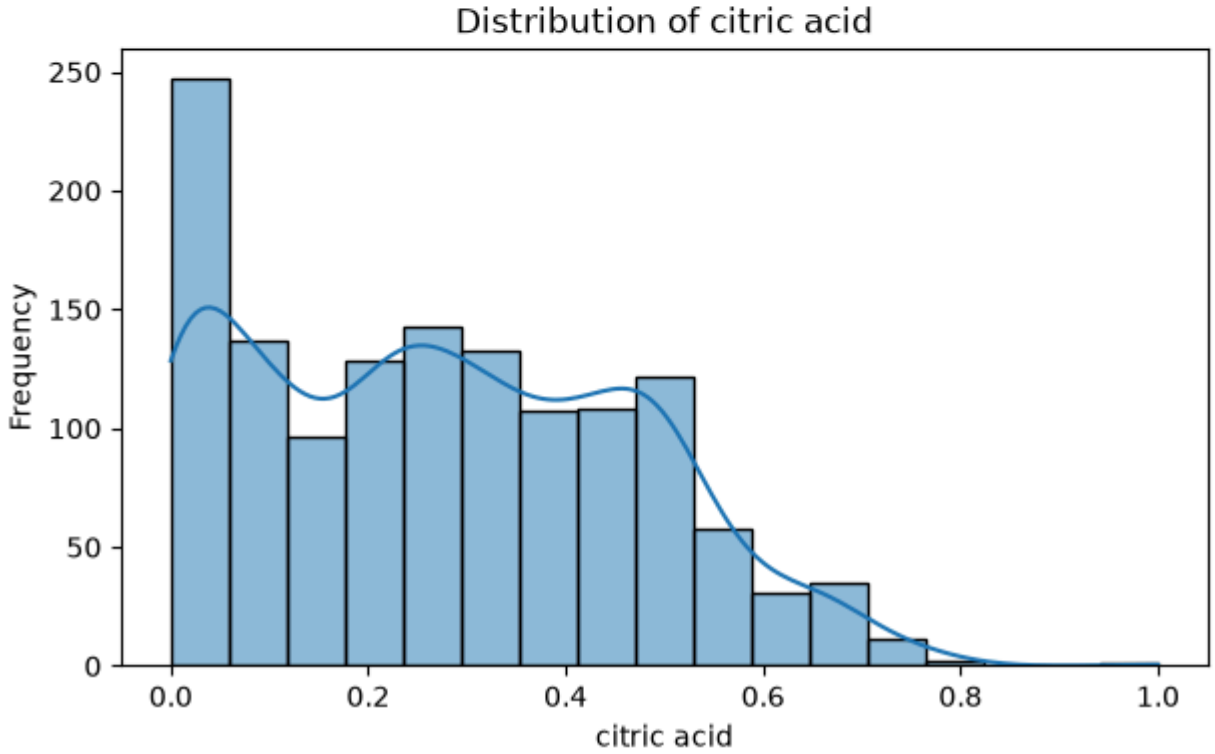
In [ ]:
```
#Histogram+KDE
import matplotlib.pyplot        as plt
import seaborn    as sns

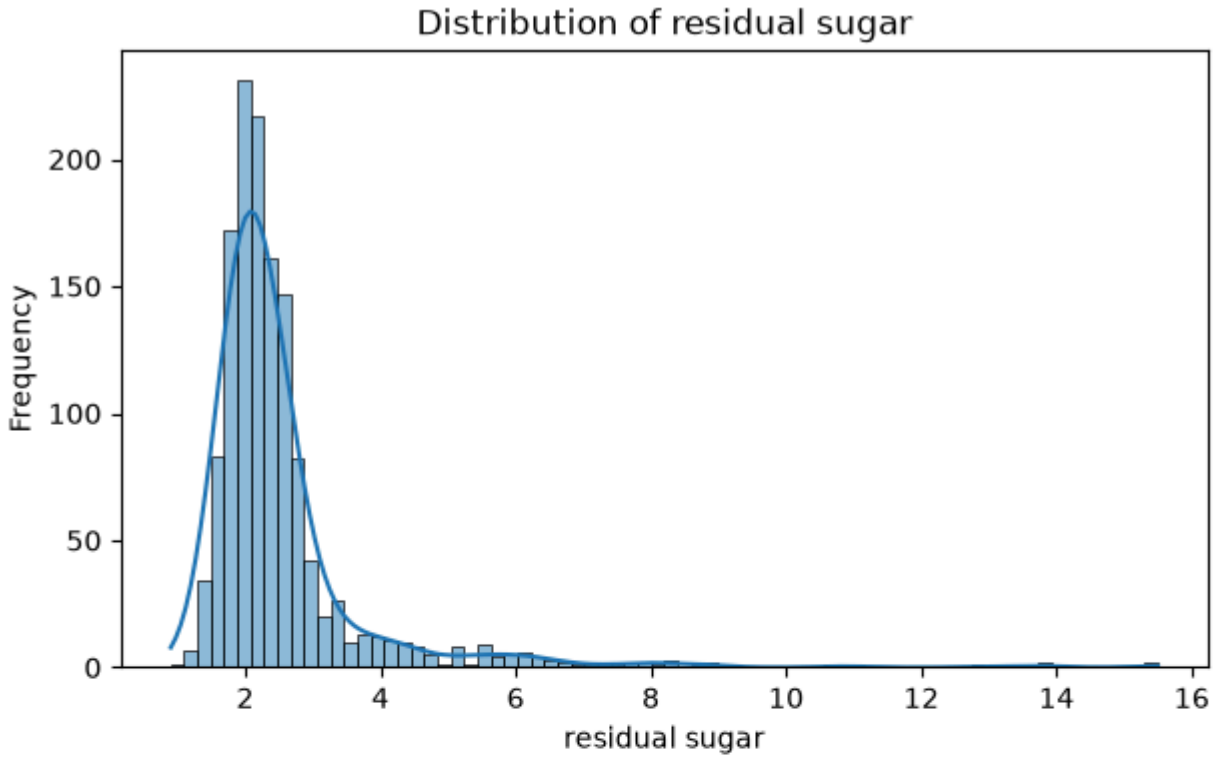
numeric_cols=df.columns
```

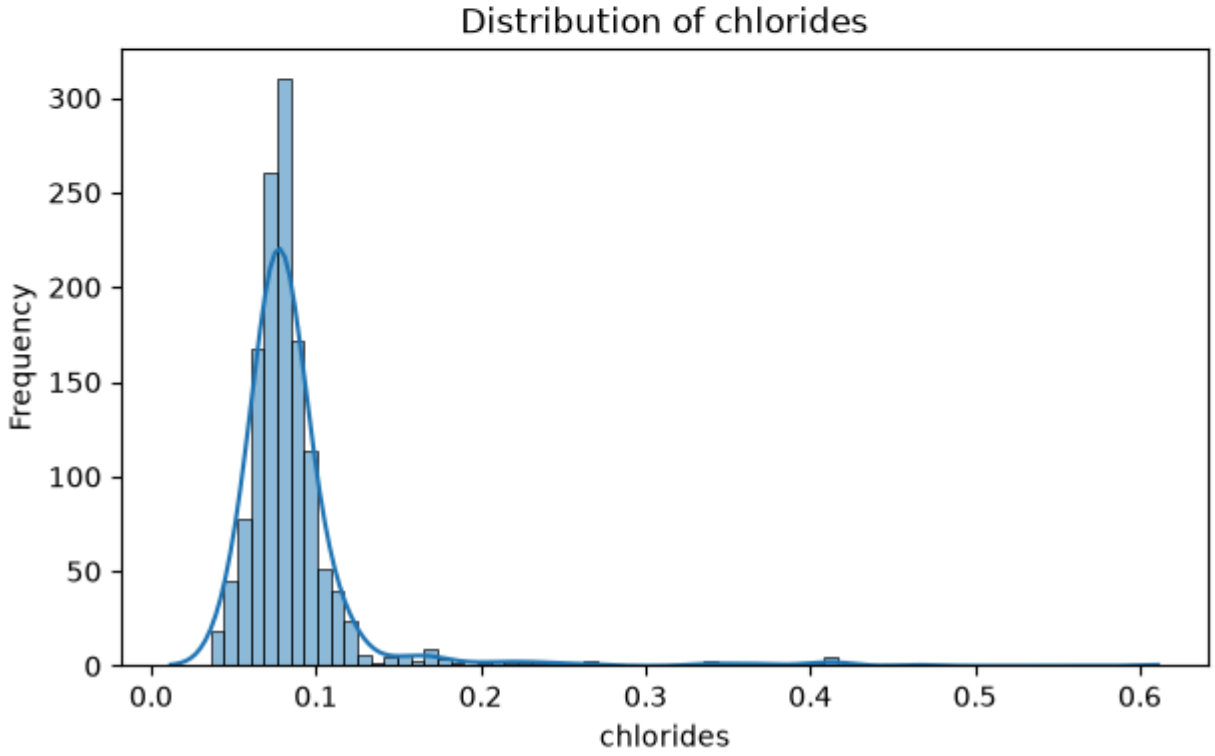
```
for col     in numeric_cols:
        plt.figure(figsize=(7,4))
        sns.histplot(df[col],kde=True)
        plt.title(f"Distribution of              {col}")
        plt.xlabel(col)
        plt.ylabel('Frequency')
        plt.show()
```

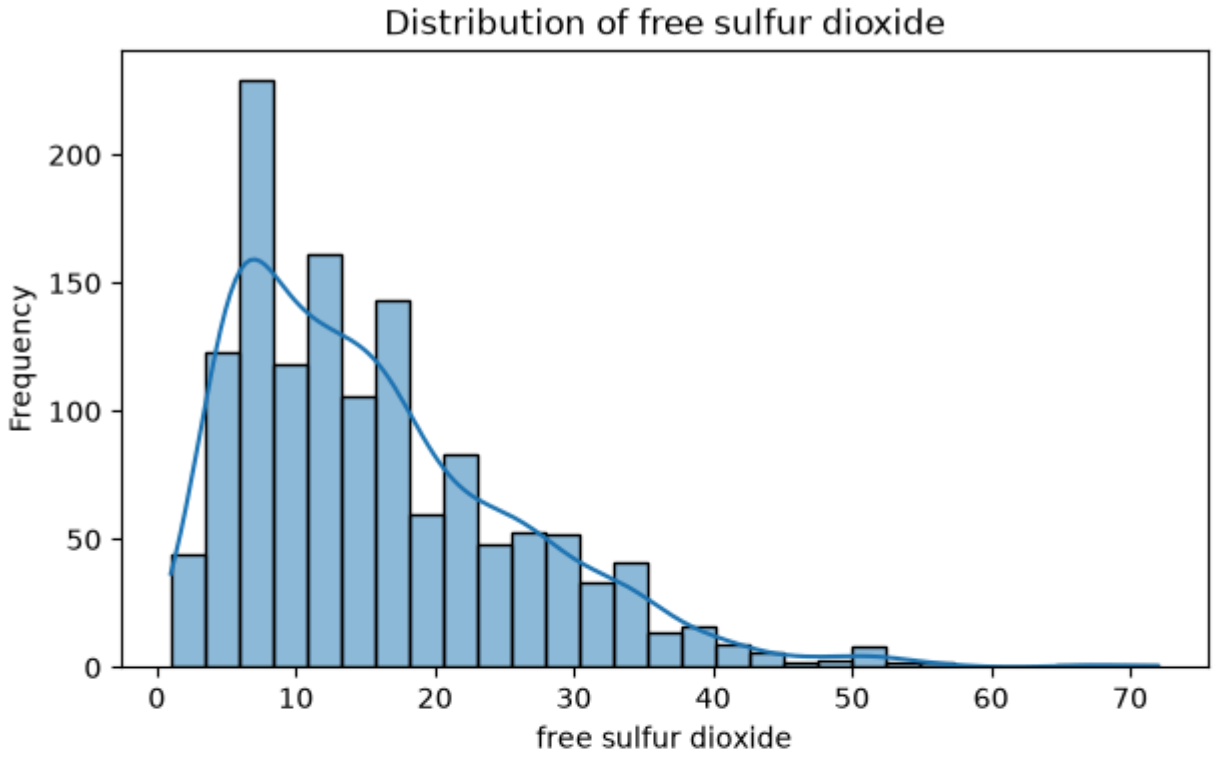


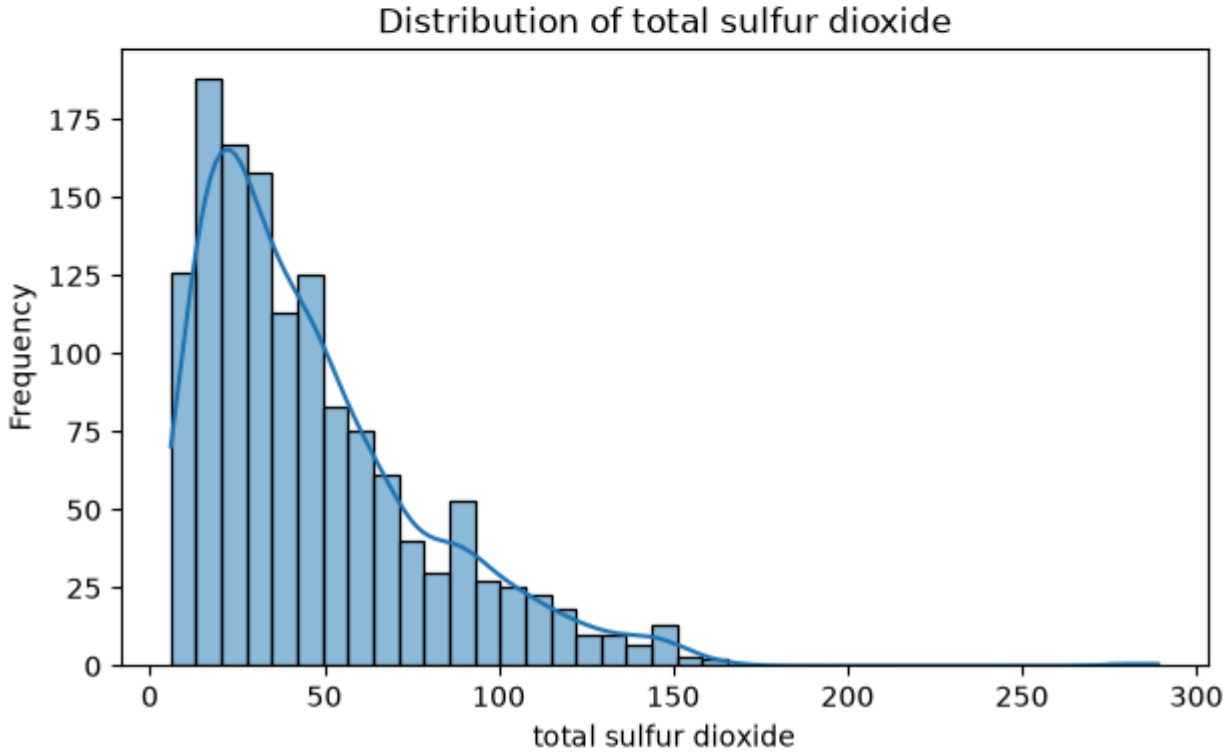

Distribution of volatile acidity
Frequency
volatile acidity
0
20
40
60
80
100
120
140
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6

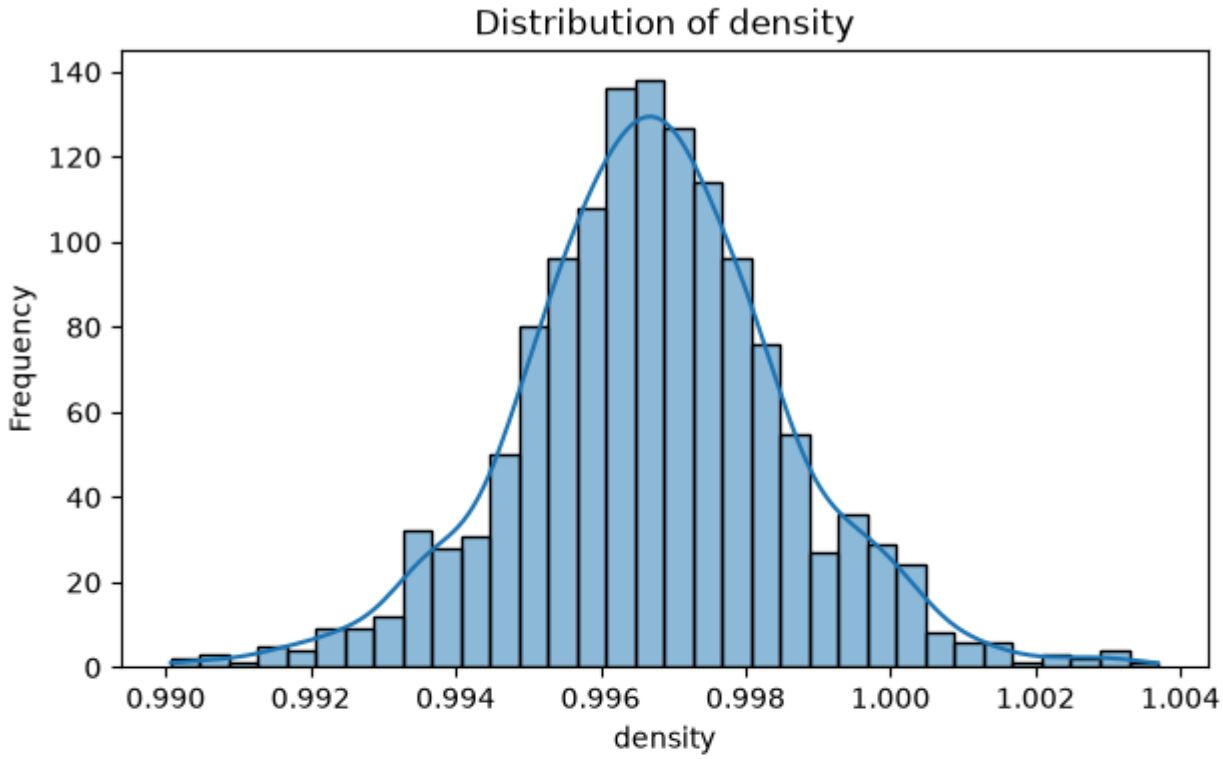

Distribution of citric acid
Frequency
citric acid
0
50
100
150
200
250
0.0
0.2
0.4
0.6
0.8
1.0

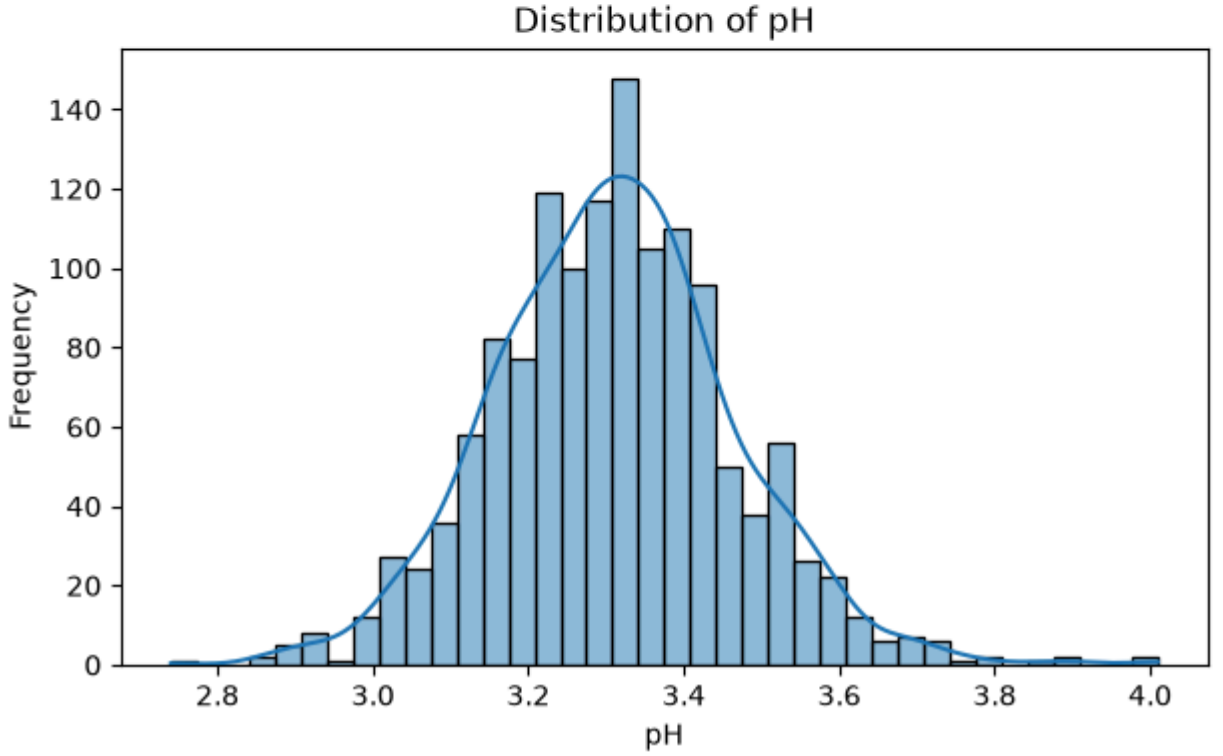

Distribution of residual sugar
Frequency
0
50
100
150
200
2
4
6
8
10
12
14
16
residual sugar

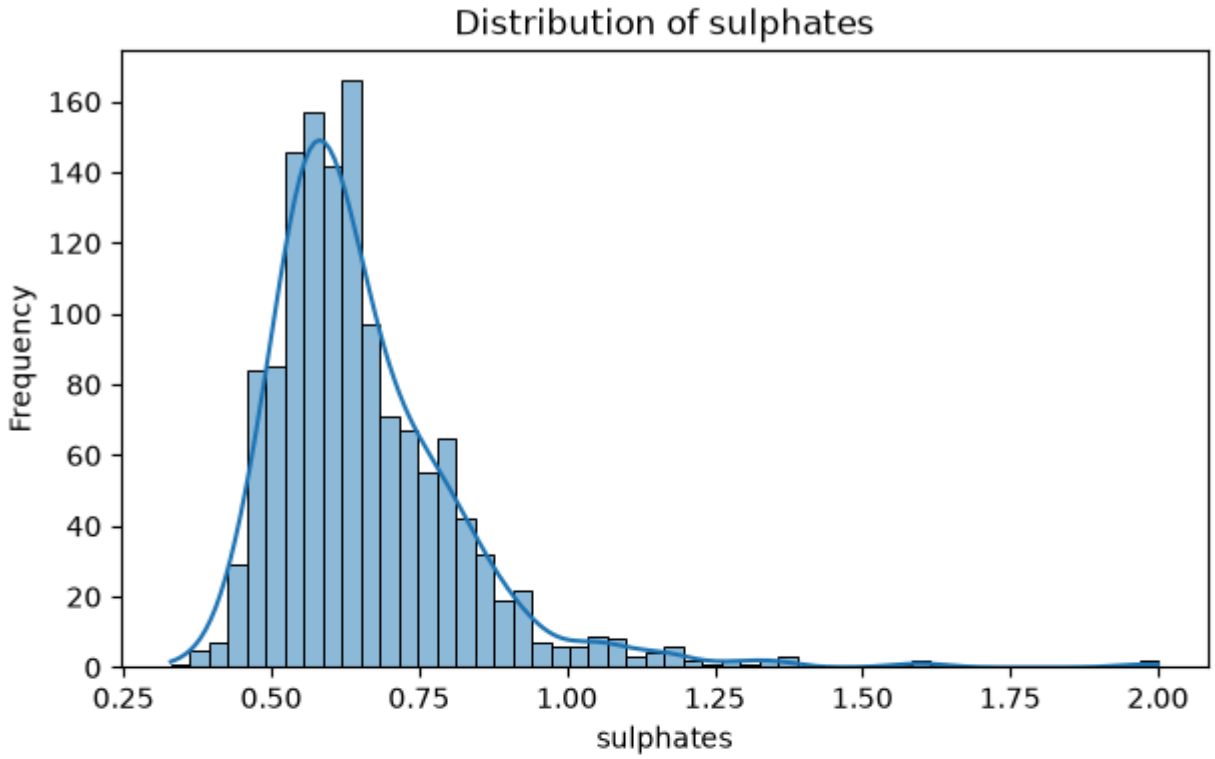

Distribution of chlorides
Frequency
0
50
100
150
200
250
300
0.0
0.1
0.2
0.3
0.4
0.5
0.6
chlorides

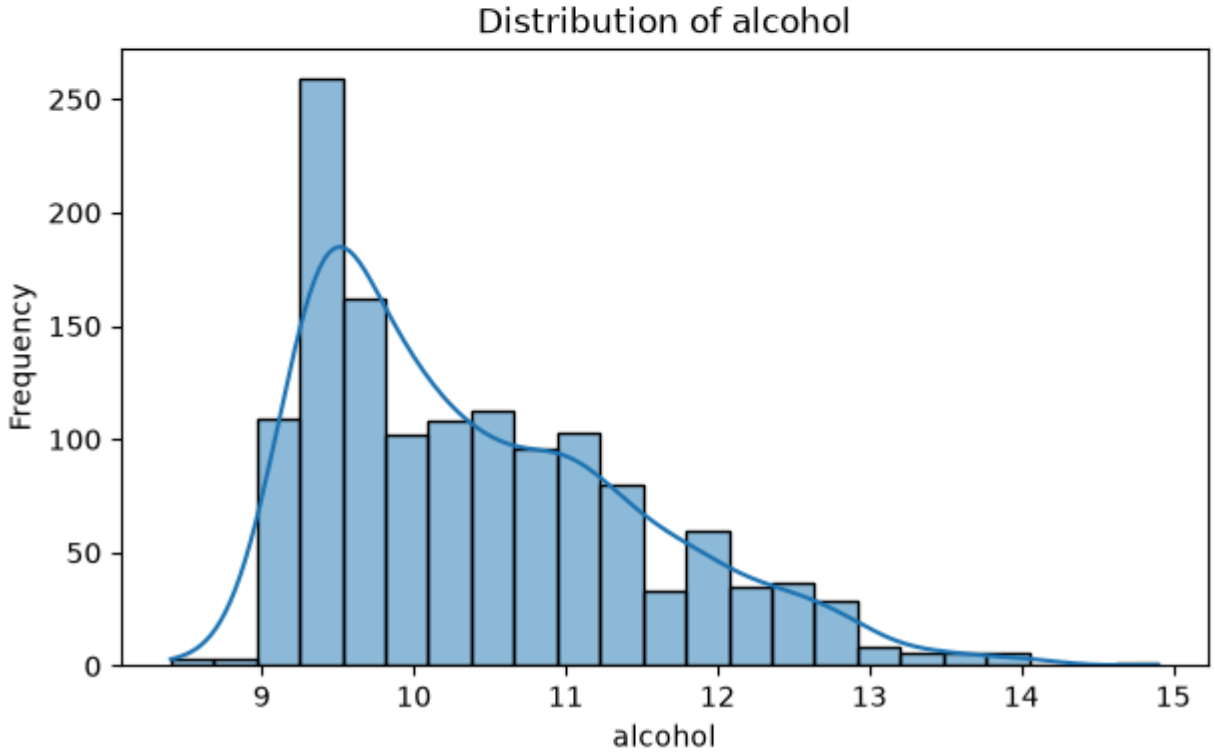

Distribution of free sulfur dioxide
Frequency
0
50
100
150
200
0
10
20
30
40
50
60
70
free sulfur dioxide

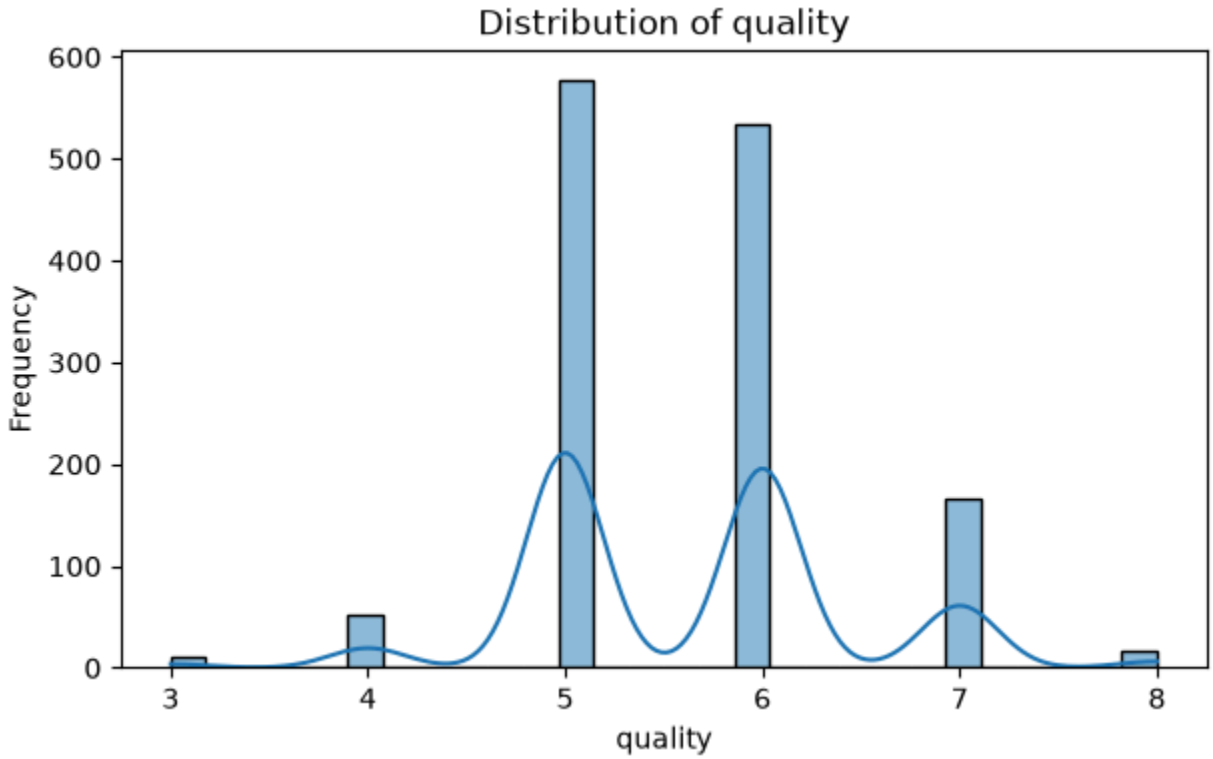

Distribution of total sulfur dioxide
Frequency
0
25
50
75
100
125
150
175
0
50
100
150
200
250
300
total sulfur dioxide


Distribution of density
Frequency
density
0
20
40
60
80
100
120
140
0.990
0.992
0.994
0.996
0.998
1.000
1.002
1.004

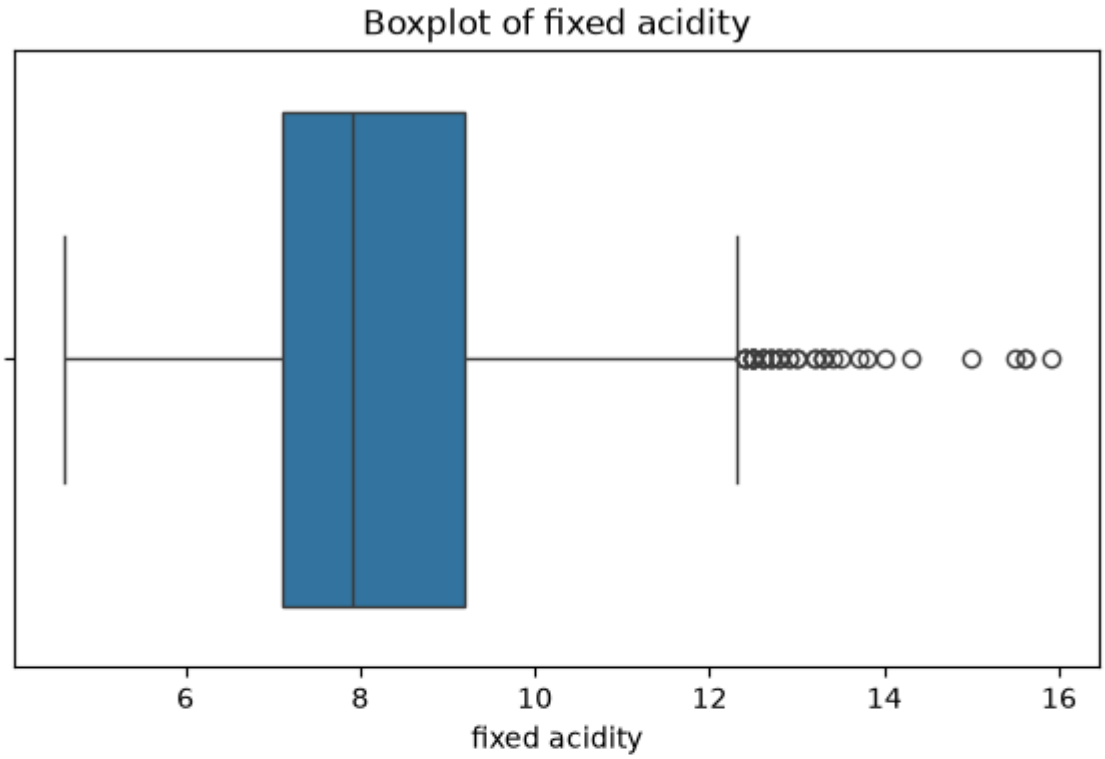

Distribution of pH
Frequency
pH
0
20
40
60
80
100
120
140
2.8
3.0
3.2
3.4
3.6
3.8
4.0

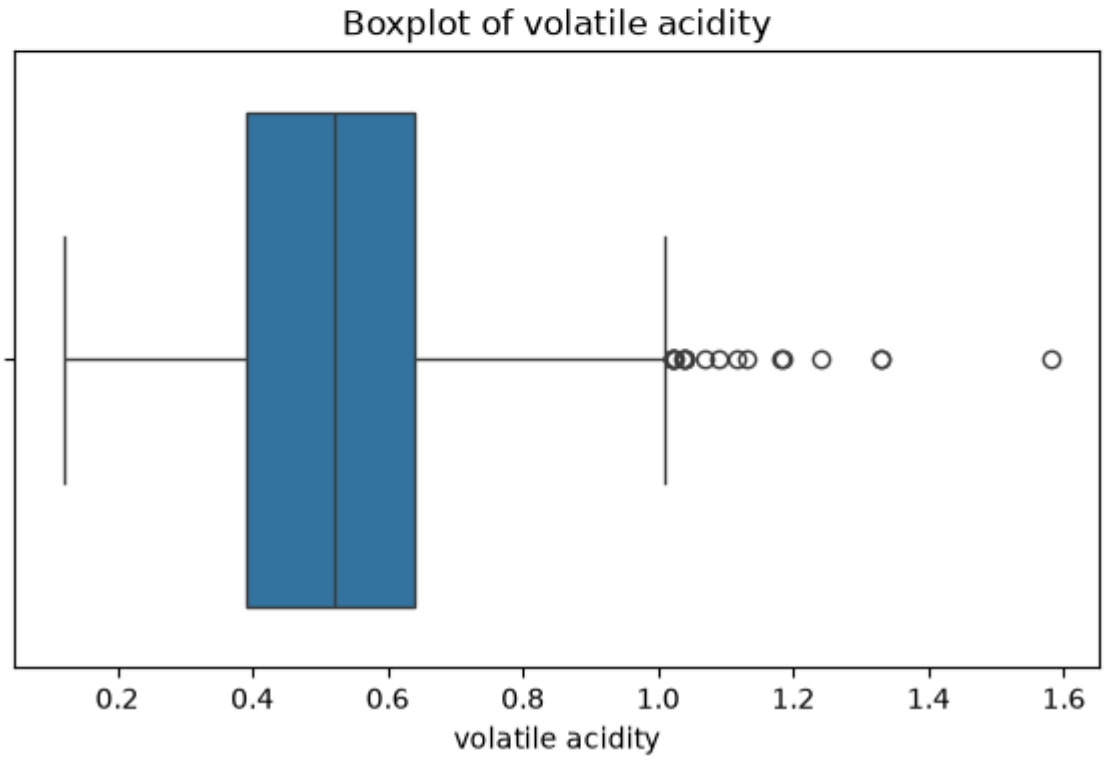

Distribution of sulphates
Frequency
0
20
40
60
80
100
120
140
160
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
sulphates

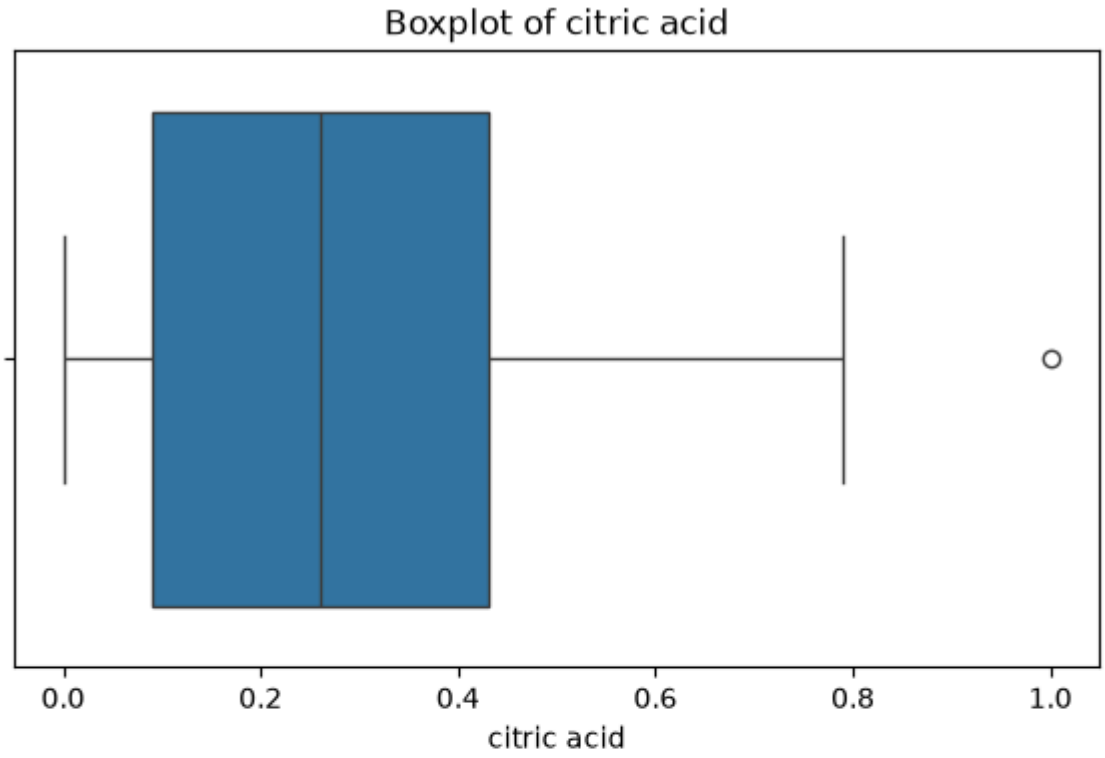

Distribution of alcohol
Frequency
0
50
100
150
200
250
9
10
11
12
13
14
15
alcohol

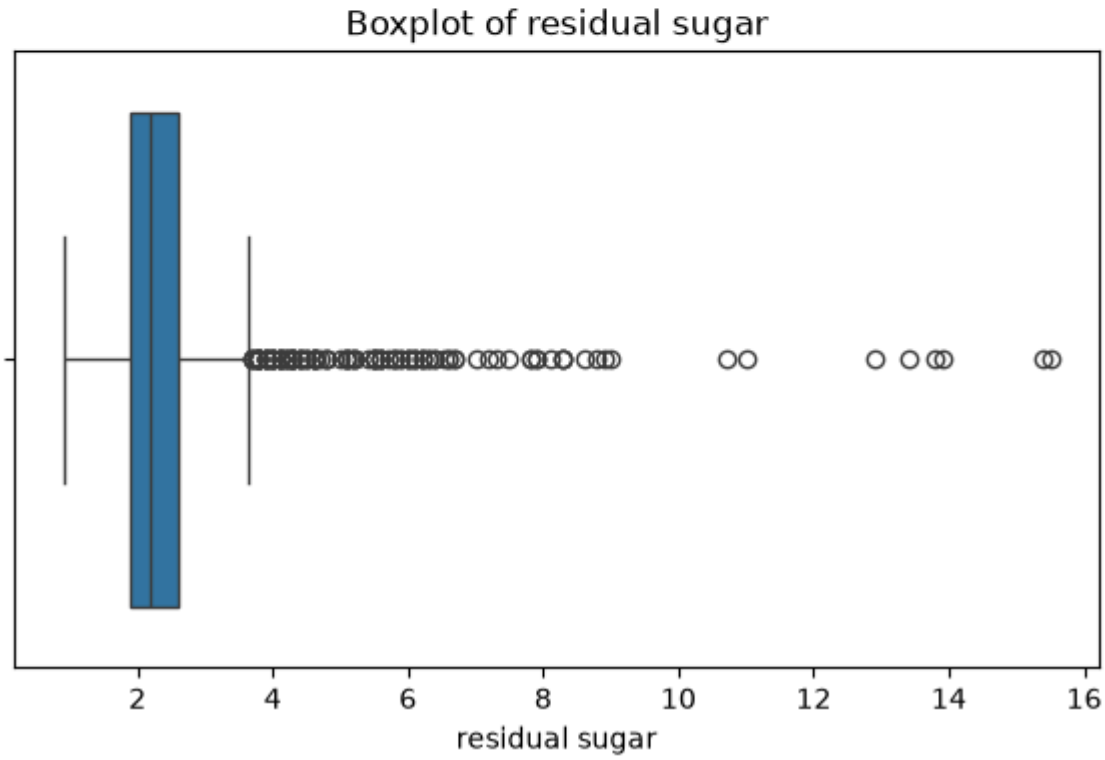


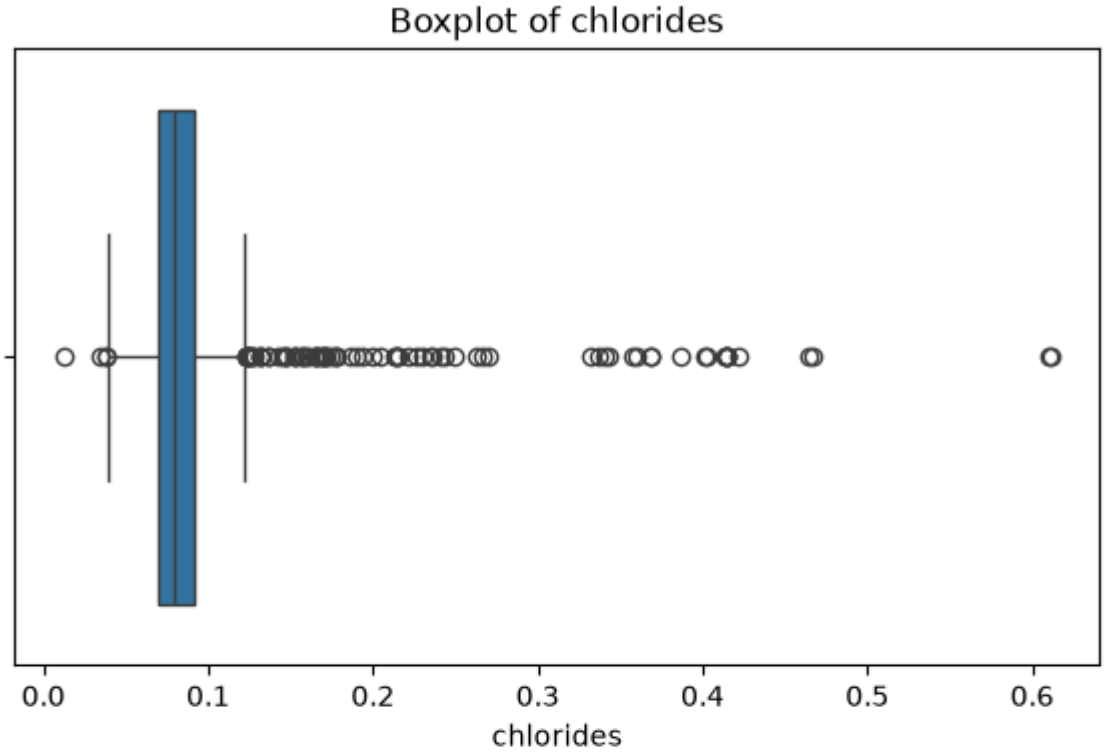
In [ ]:
```
#boxplot-Outlier Analysis
for col     in numeric_cols:
        plt.figure(figsize=(7,4))
        sns.boxplot(x=df[col])
        plt.title(f"Boxplot of plt.show{col}")
```

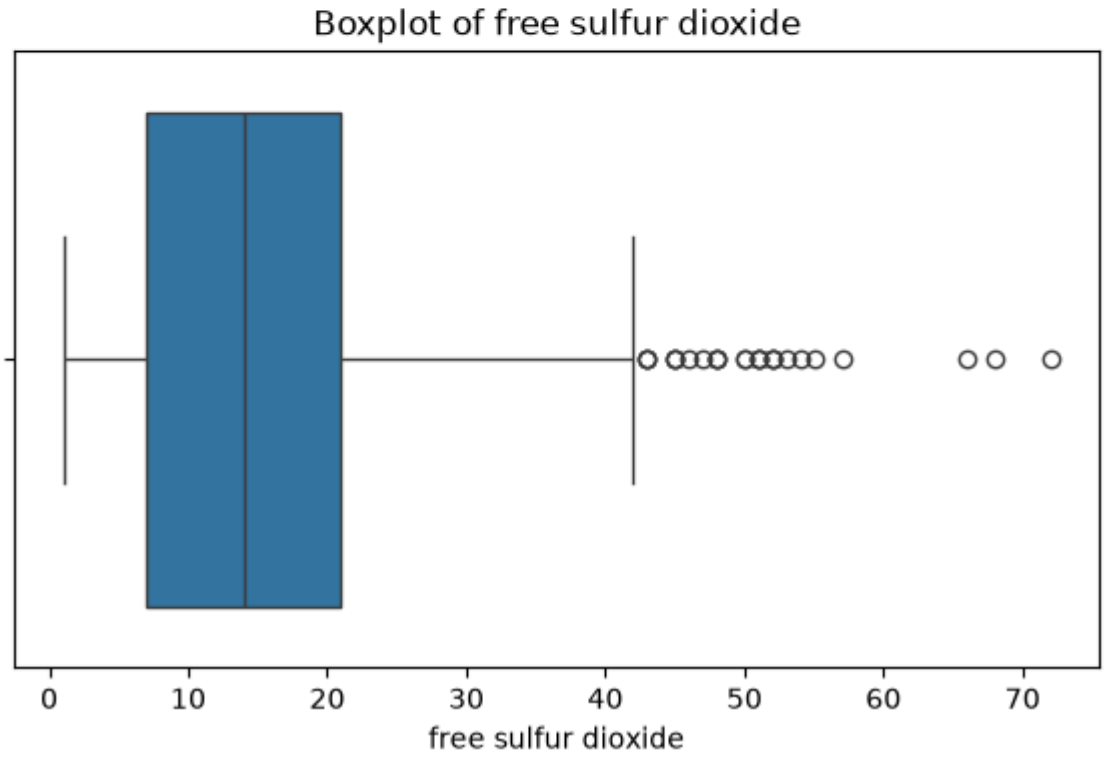

Boxplot of fixed acidity
6
8
10
12
14
16
fixed acidity

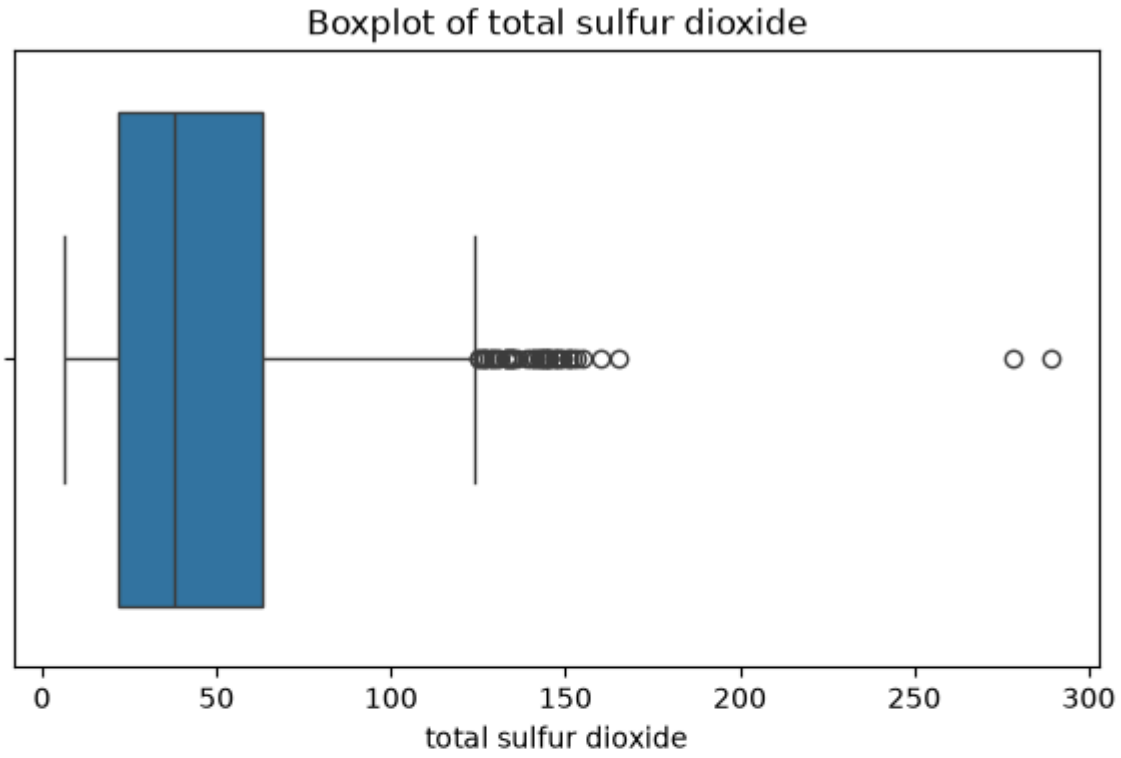

Boxplot of volatile acidity
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
volatile acidity

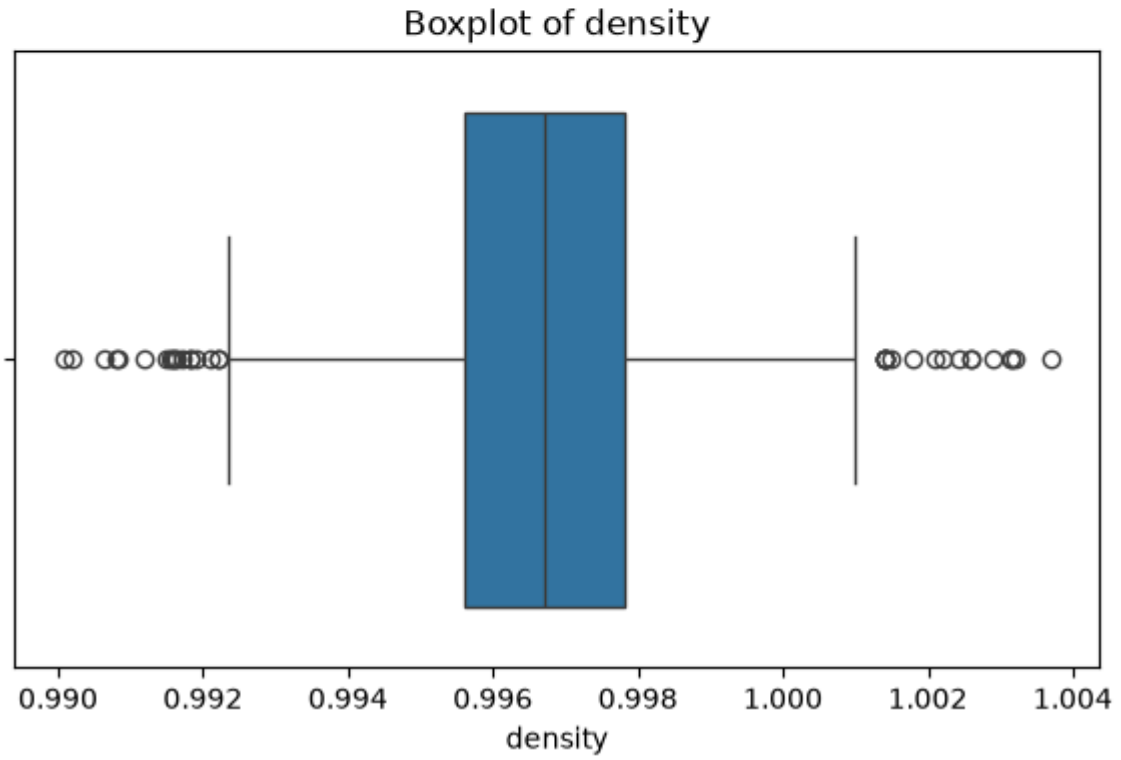

Boxplot of citric acid
0.0
0.2
0.4
0.6
0.8
1.0
citric acid

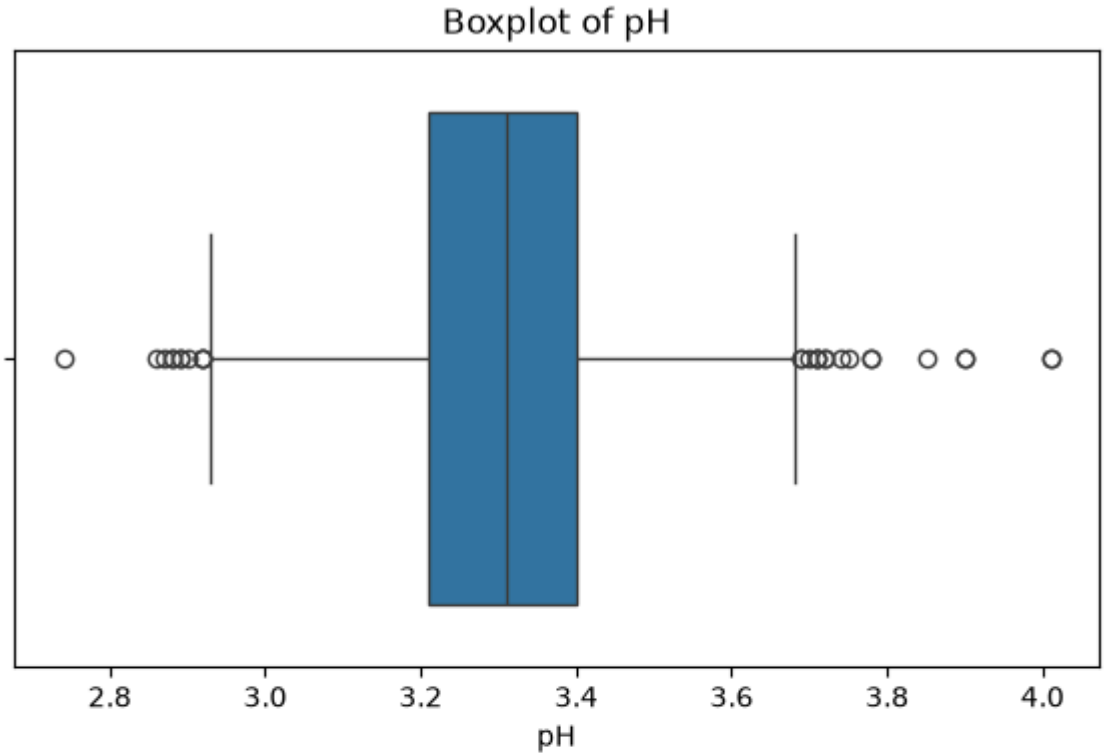

Boxplot of residual sugar
2
4
6
8
10
12
14
16
residual sugar

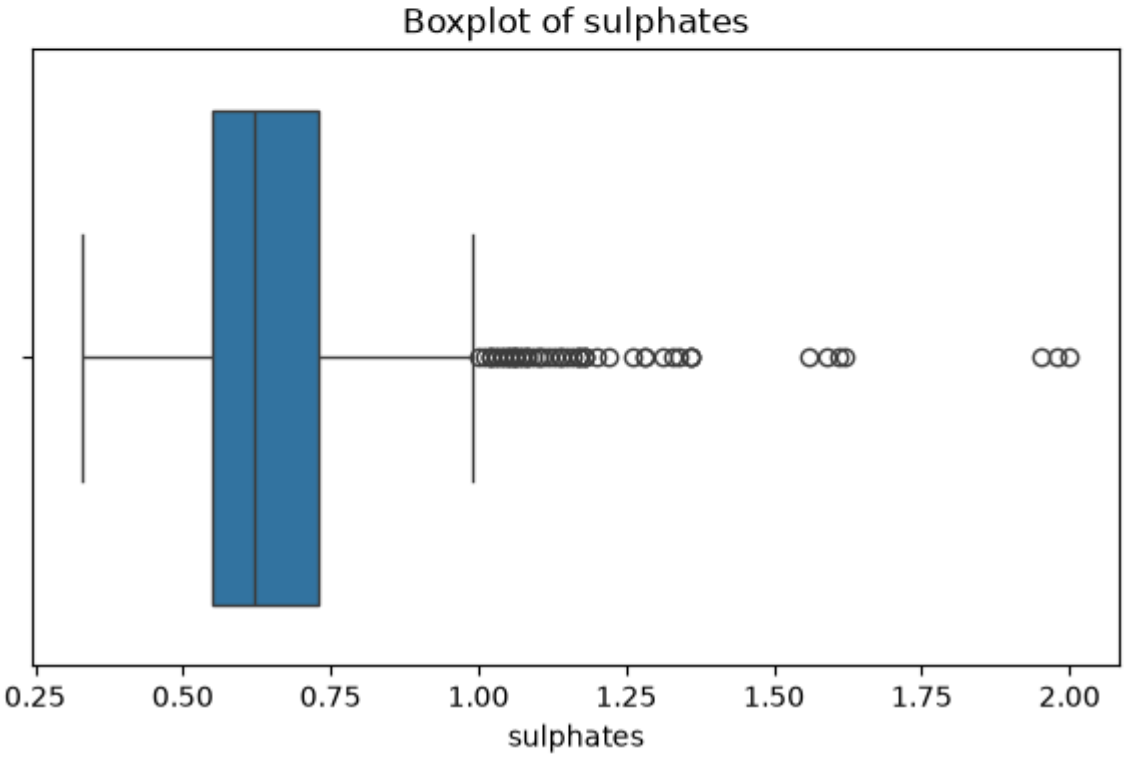

Boxplot of chlorides
0.0
0.1
0.2
0.3
0.4
0.5
0.6
chlorides

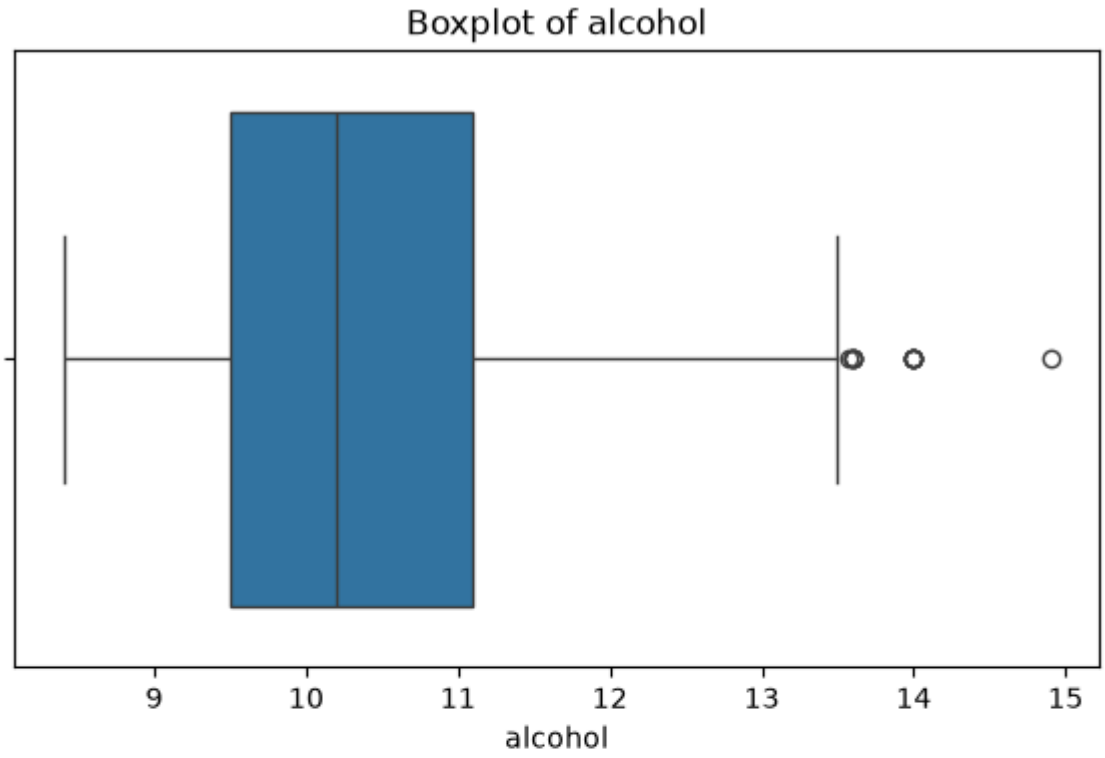

Boxplot of free sulfur dioxide
0
10
20
30
40
50
60
70
free sulfur dioxide

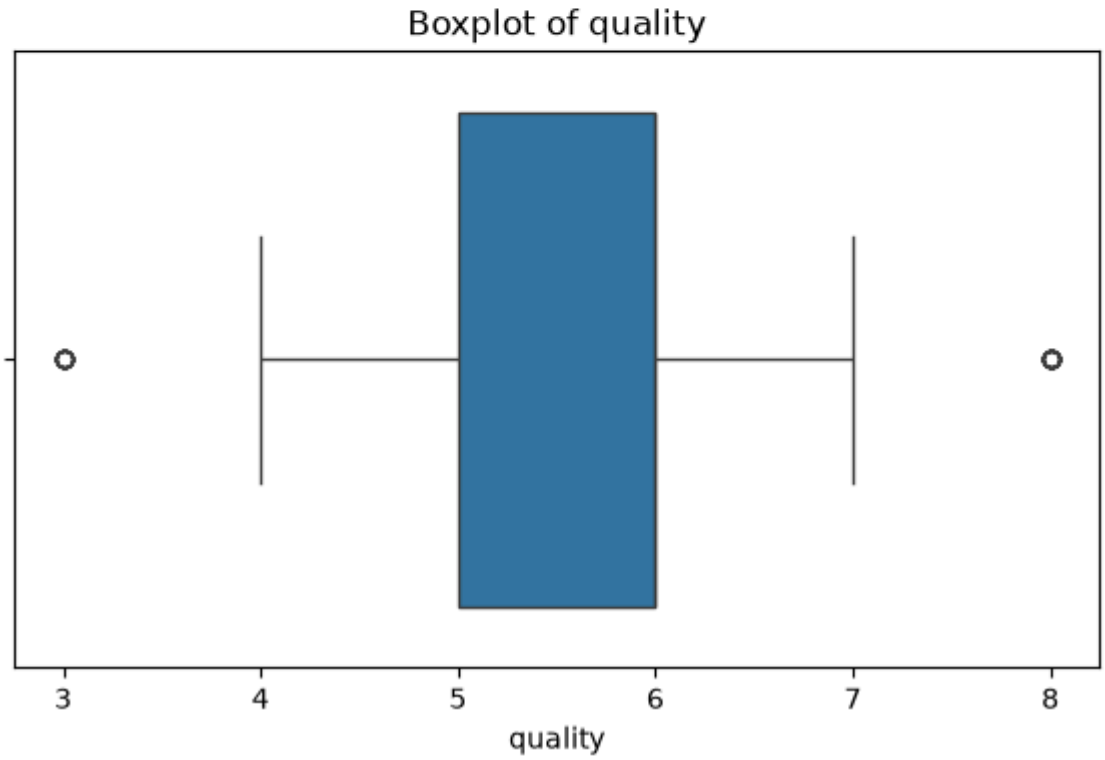

Boxplot of total sulfur dioxide
0
50
100
150
200
250
300
total sulfur dioxide


Boxplot of density
0.990
0.992
0.994
0.996
0.998
1.000
1.002
1.004
density


Boxplot of pH
2.8
3.0
3.2
3.4
3.6
3.8
4.0
pH

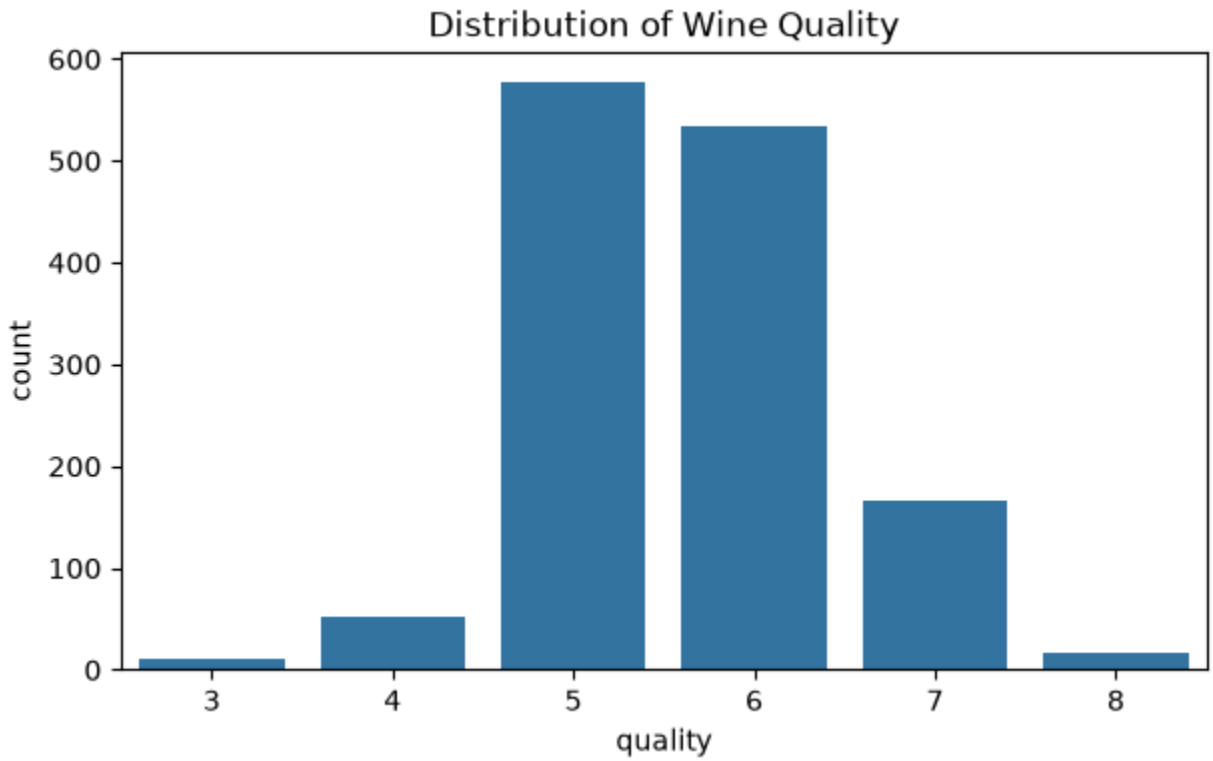

Boxplot of sulphates
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
sulphates





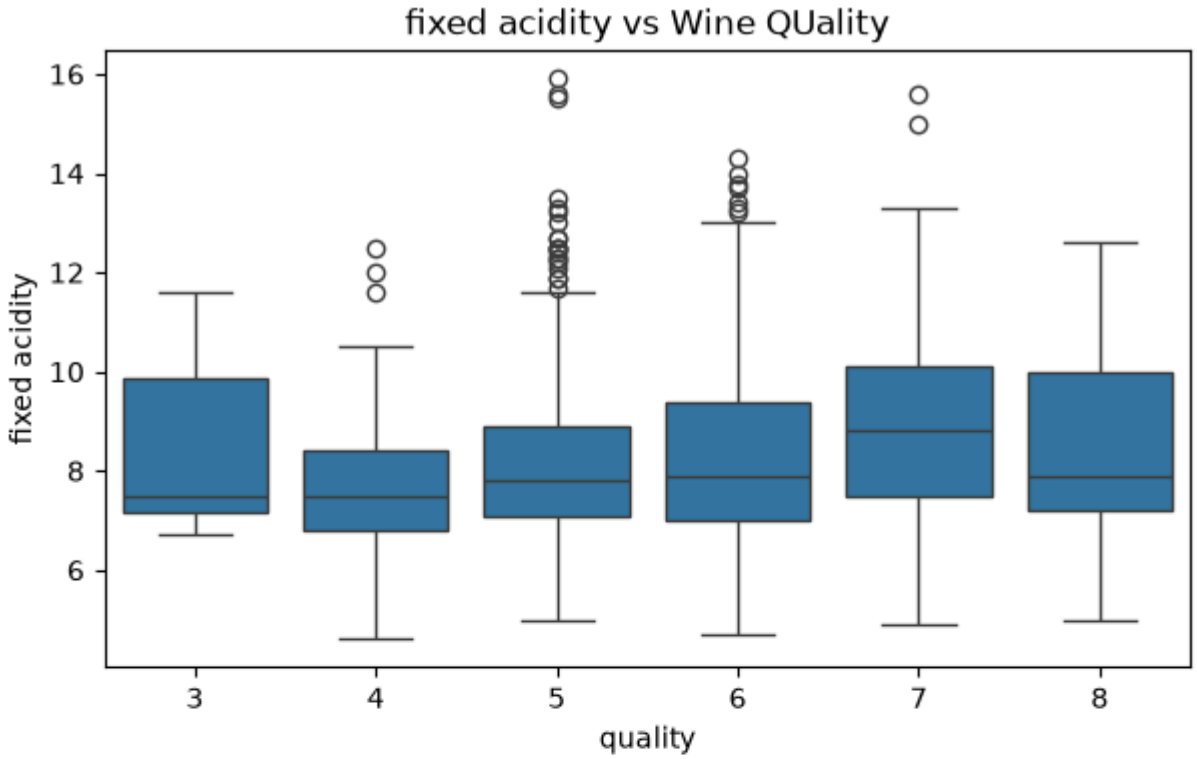
In [ ]:

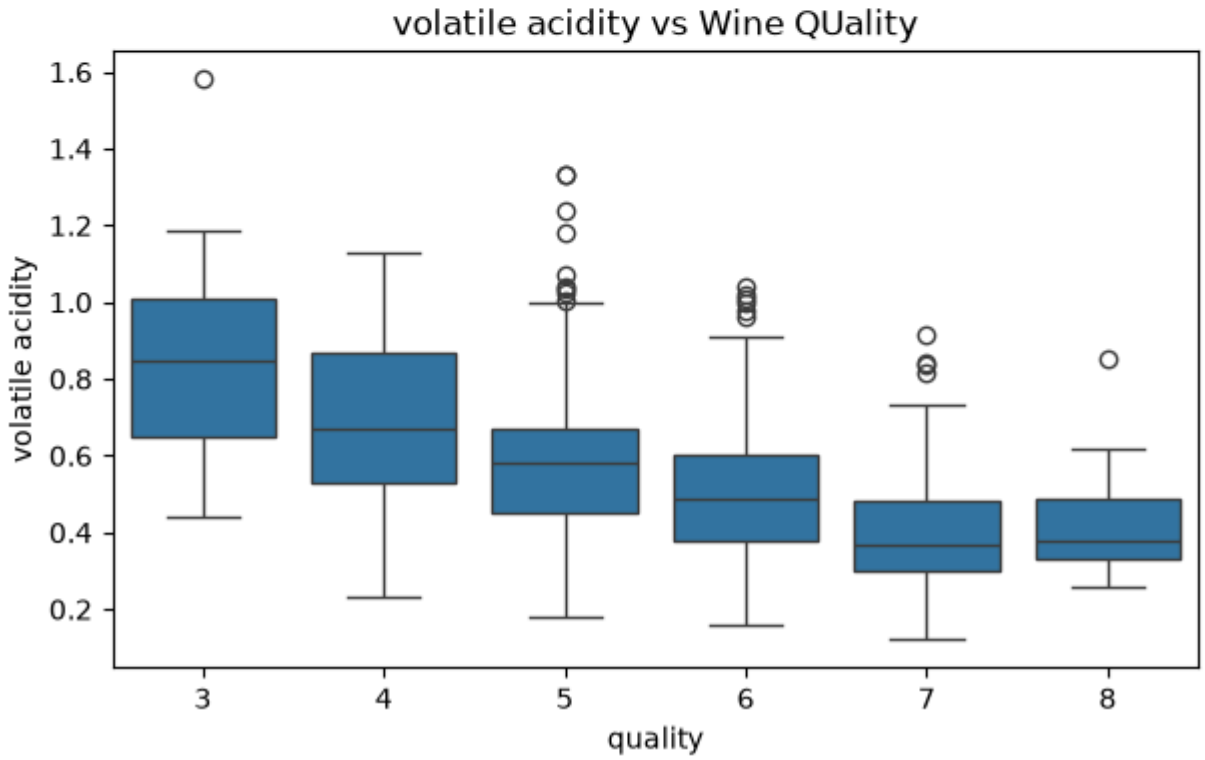
```
#Quality Distribution
    plt.figure(figsize=(7,4))
    sns.countplot(x='quality',data=df)
    plt.title(f"Distribution of Wine Quality")
    plt.show()
```

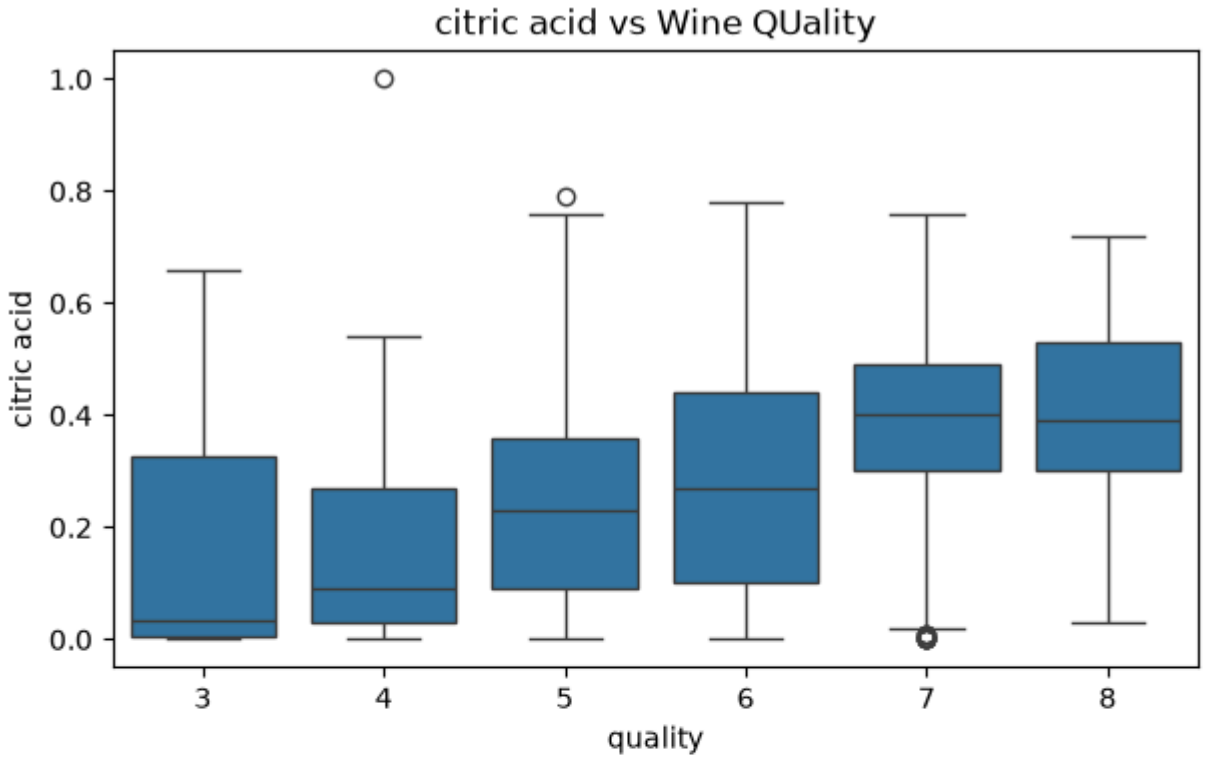


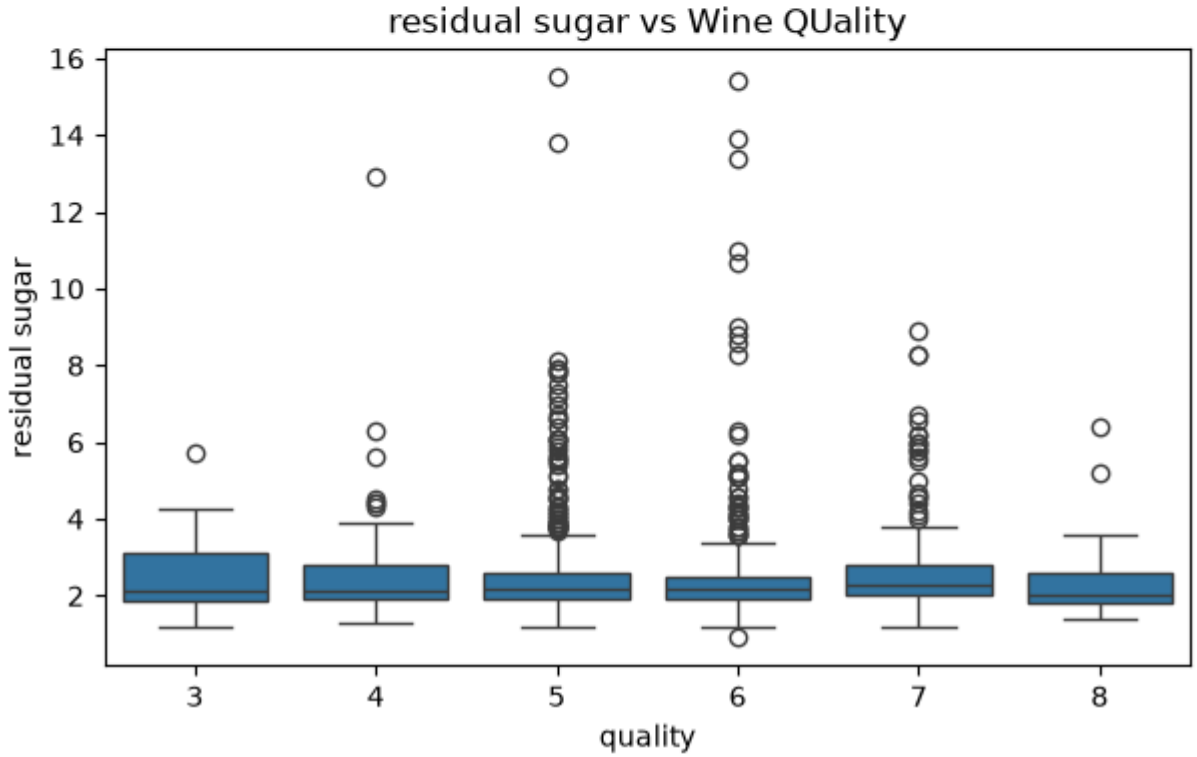
In [ ]:

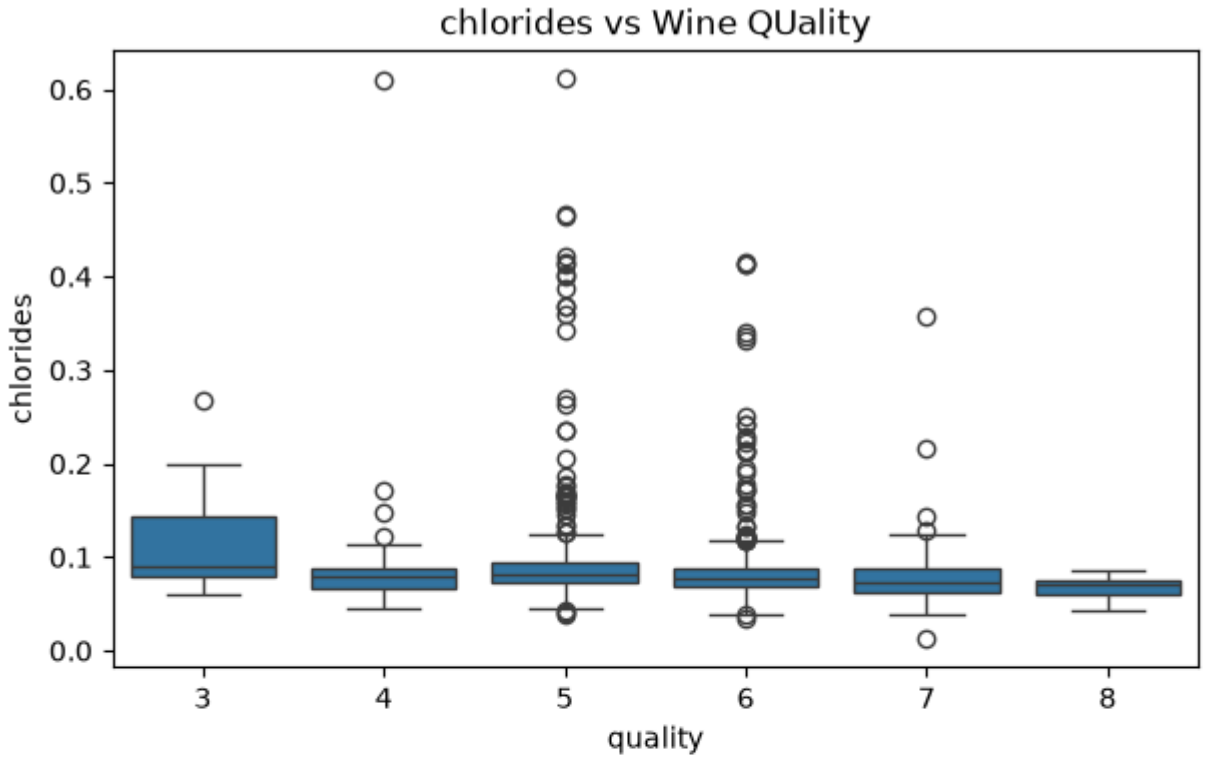
```
features=['fixed acidity',          'volatile acidity',          'citric acid',              'residual s
          'chlorides',       'free sulfur dioxide',          'total sulfur dioxide',             'densi
          'pH', 'sulphates', 'alcohol', 'quality']
for col     in features:
      plt.figure(figsize=(7,4))
      sns.boxplot(x='quality',y=col,data=df)
      plt.title(f'{col}          vs Wine QUality')
      plt.show()
```

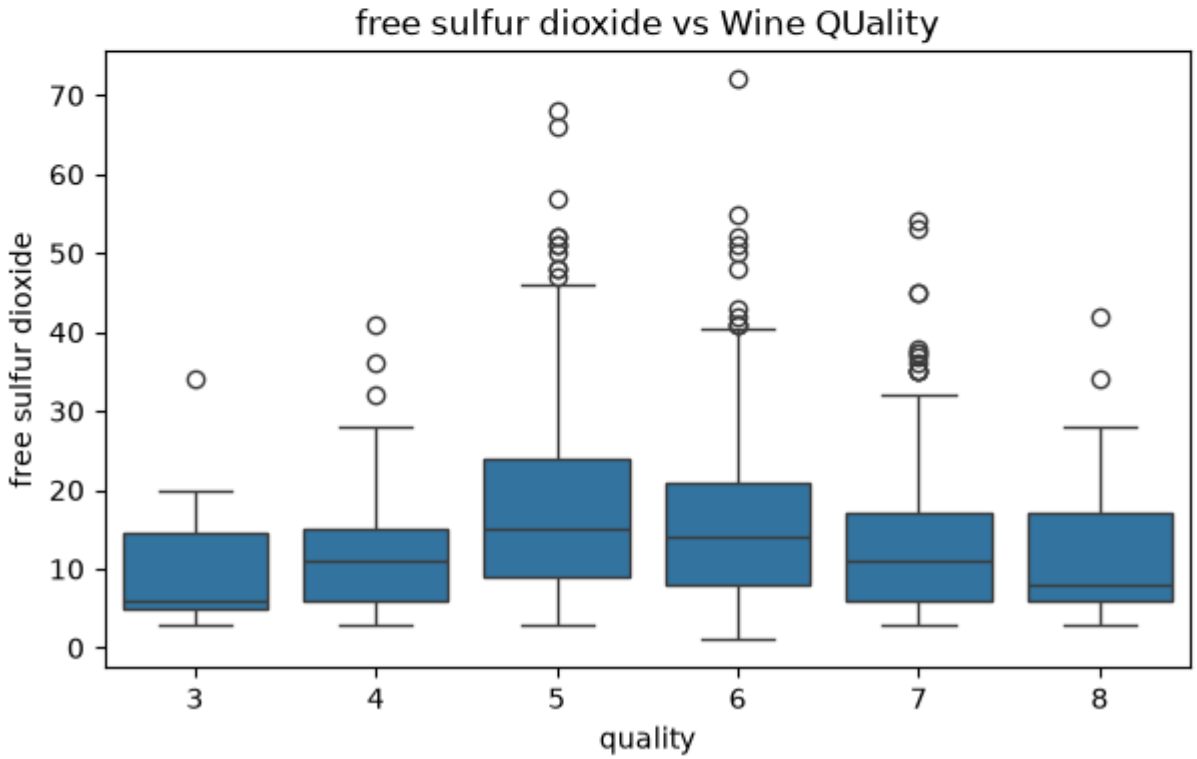

fixed acidity vs Wine QUality
fixed acidity
16
14
12
10
8
6
3
4
5
6
7
8
quality

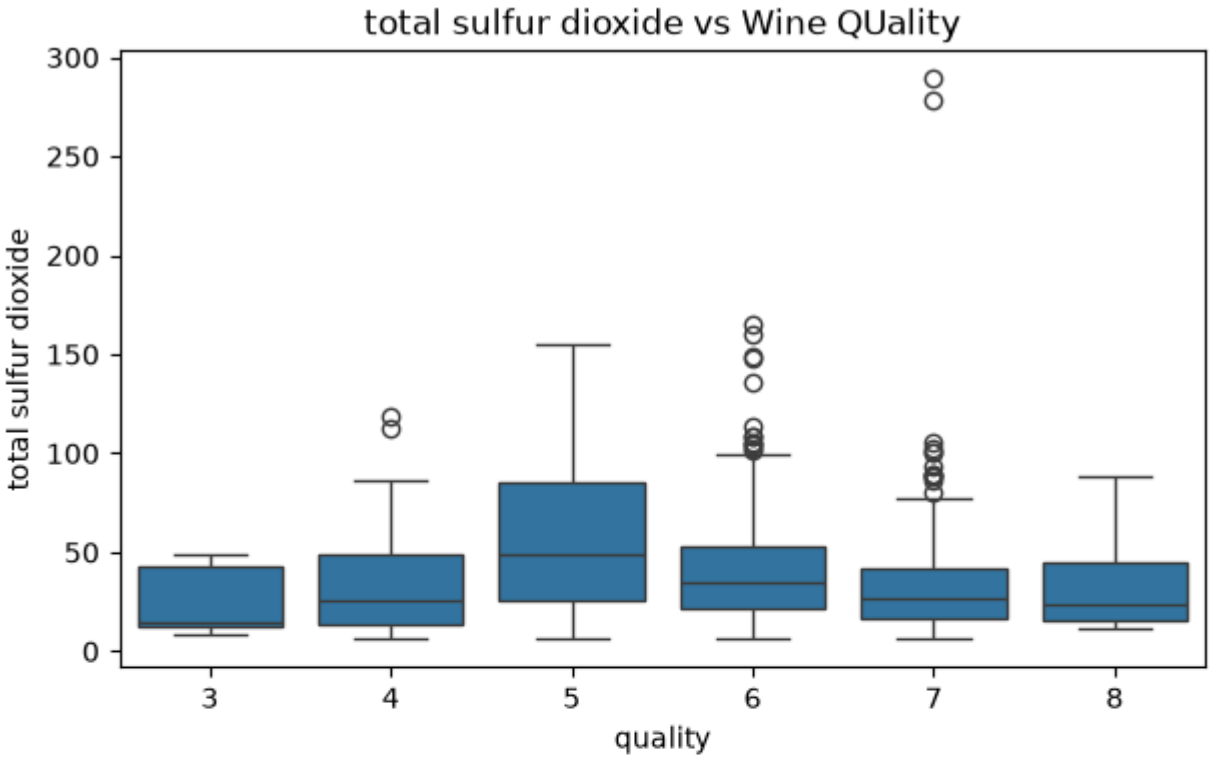

volatile acidity vs Wine QUality
volatile acidity
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
3
4
5
6
7
8
quality

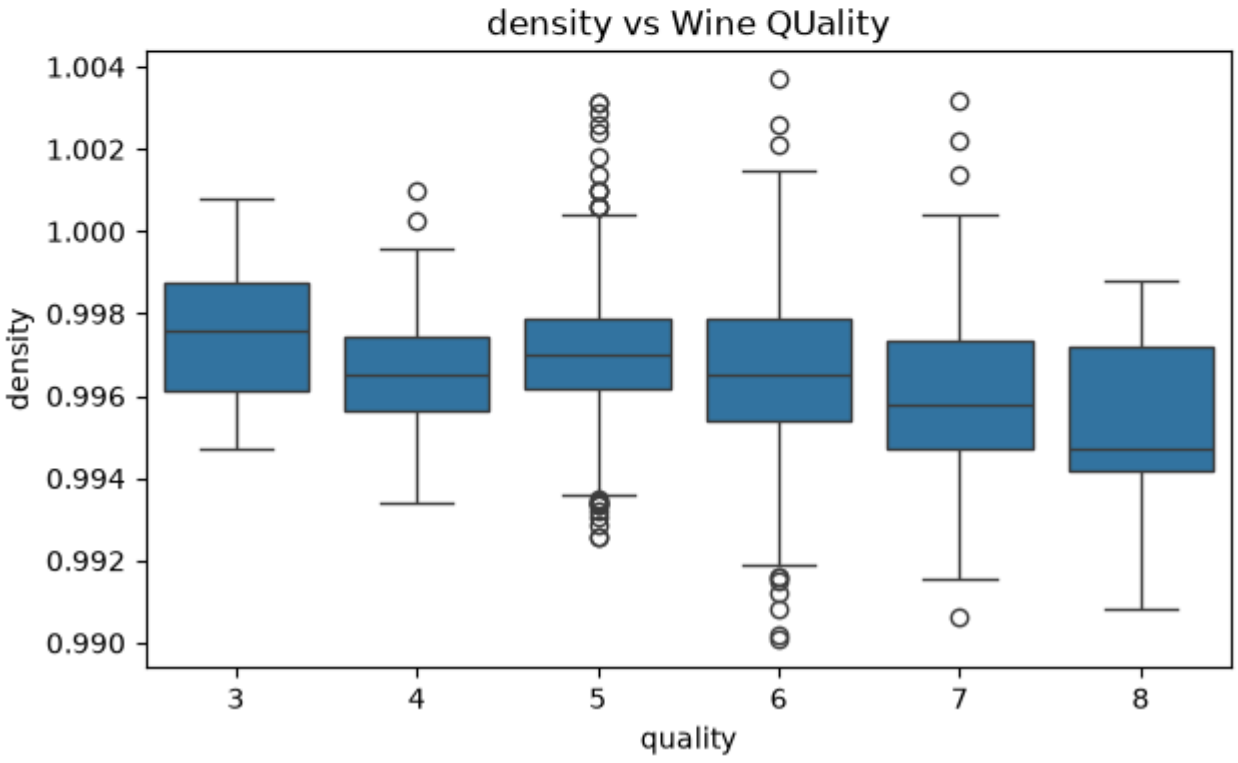

citric acid vs Wine QUality
citric acid
1.0
0.8
0.6
0.4
0.2
0.0
3
4
5
6
7
8
quality

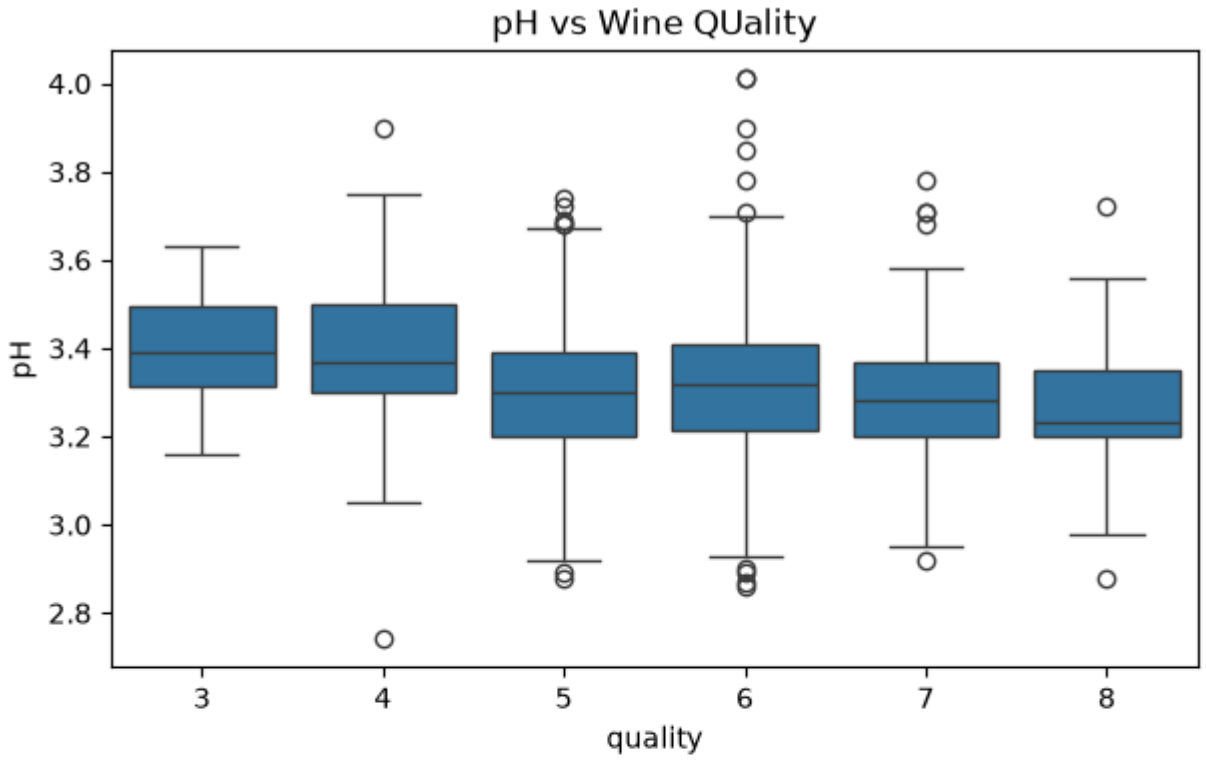

residual sugar vs Wine QUality
residual sugar
16
14
12
10
8
6
4
2
3
4
5
6
7
8
quality

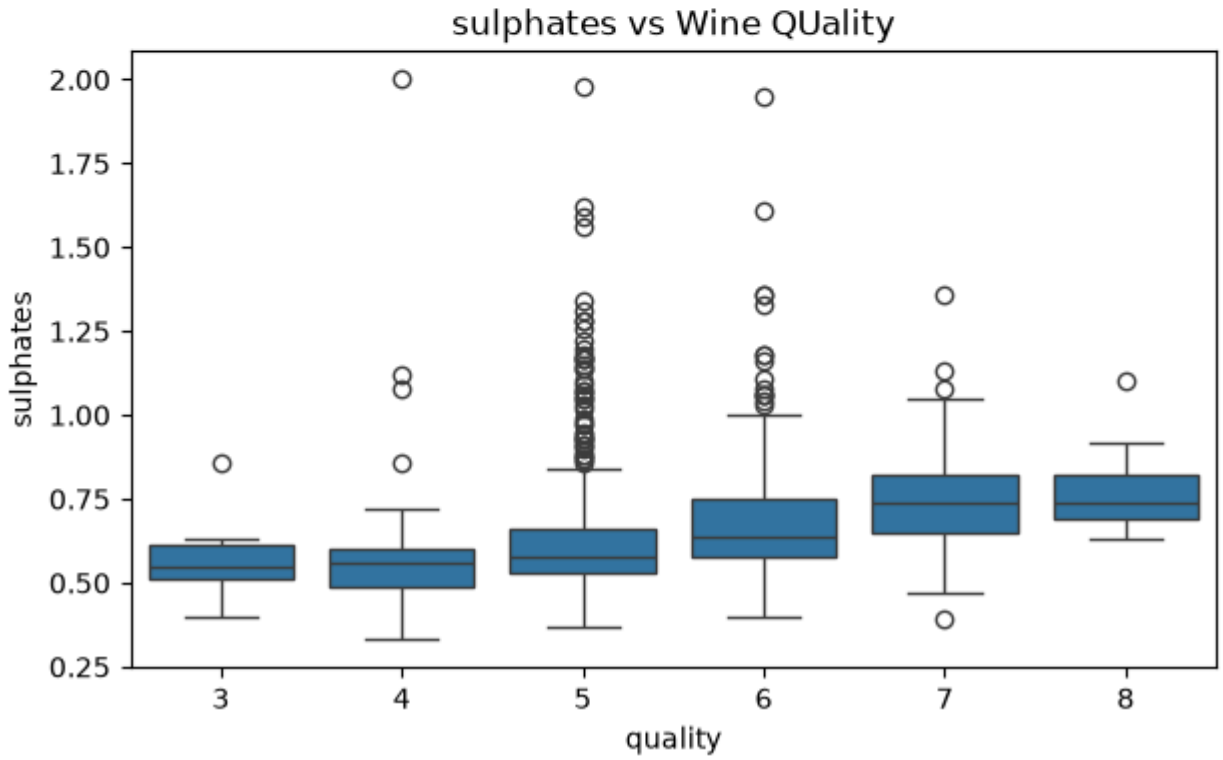

chlorides vs Wine QUality
chlorides
0.0
0.1
0.2
0.3
0.4
0.5
0.6
3
4
5
6
7
8
quality

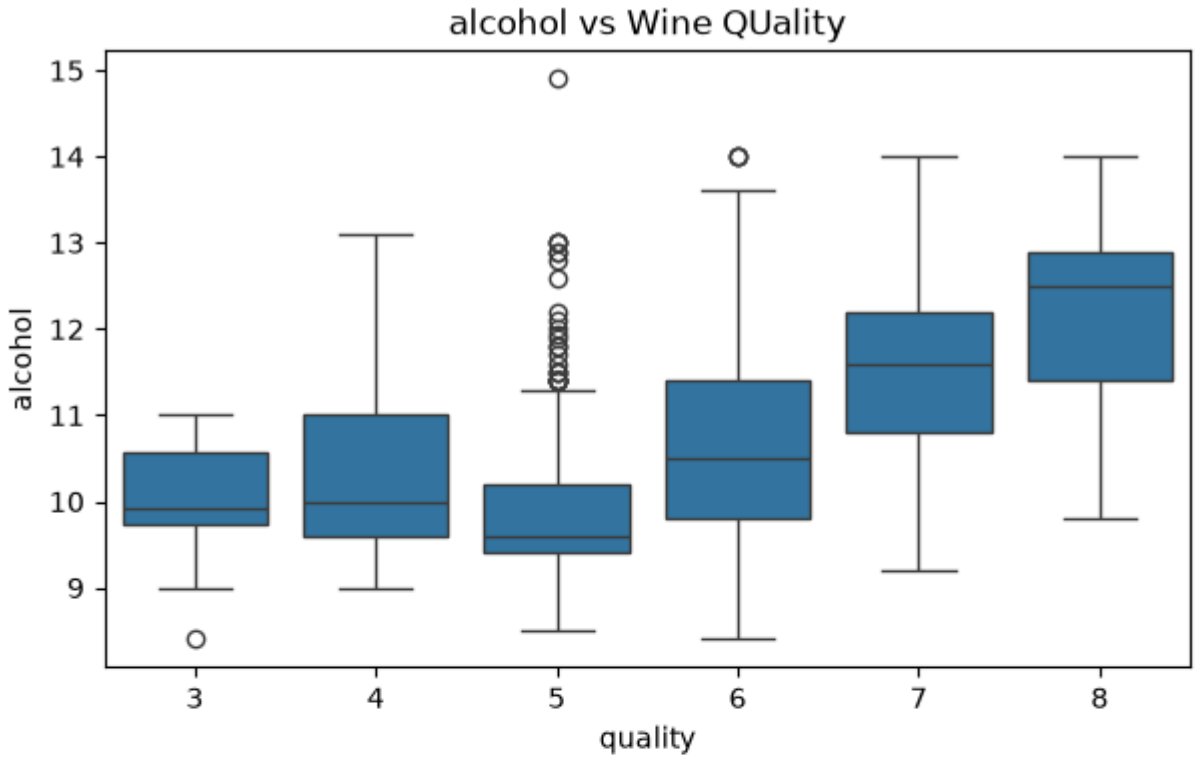

free sulfur dioxide vs Wine QUality
free sulfur dioxide
0
10
20
30
40
50
60
70
3
4
5
6
7
8
quality

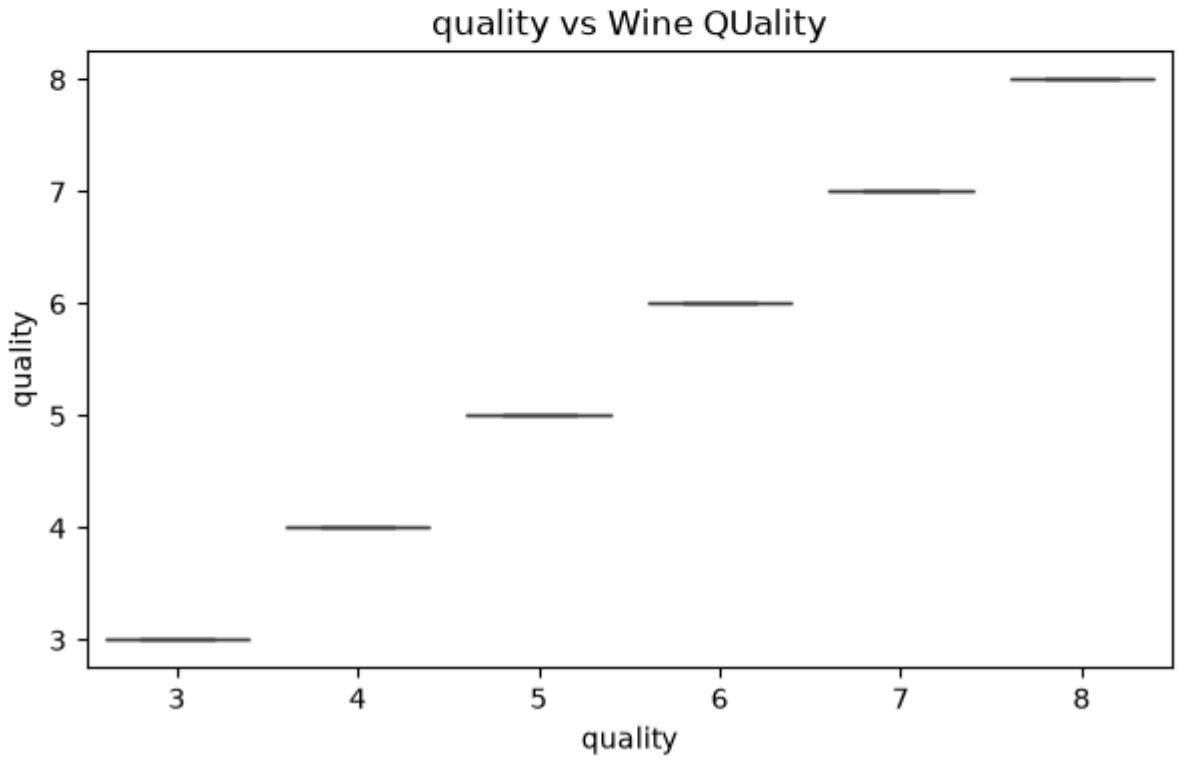

total sulfur dioxide vs Wine QUality
total sulfur dioxide
300
250
200
150
100
50
0
3
4
5
6
7
8
quality


density vs Wine QUality
density
1.004
1.002
1.000
0.998
0.996
0.994
0.992
0.990
3
4
5
6
7
8
quality

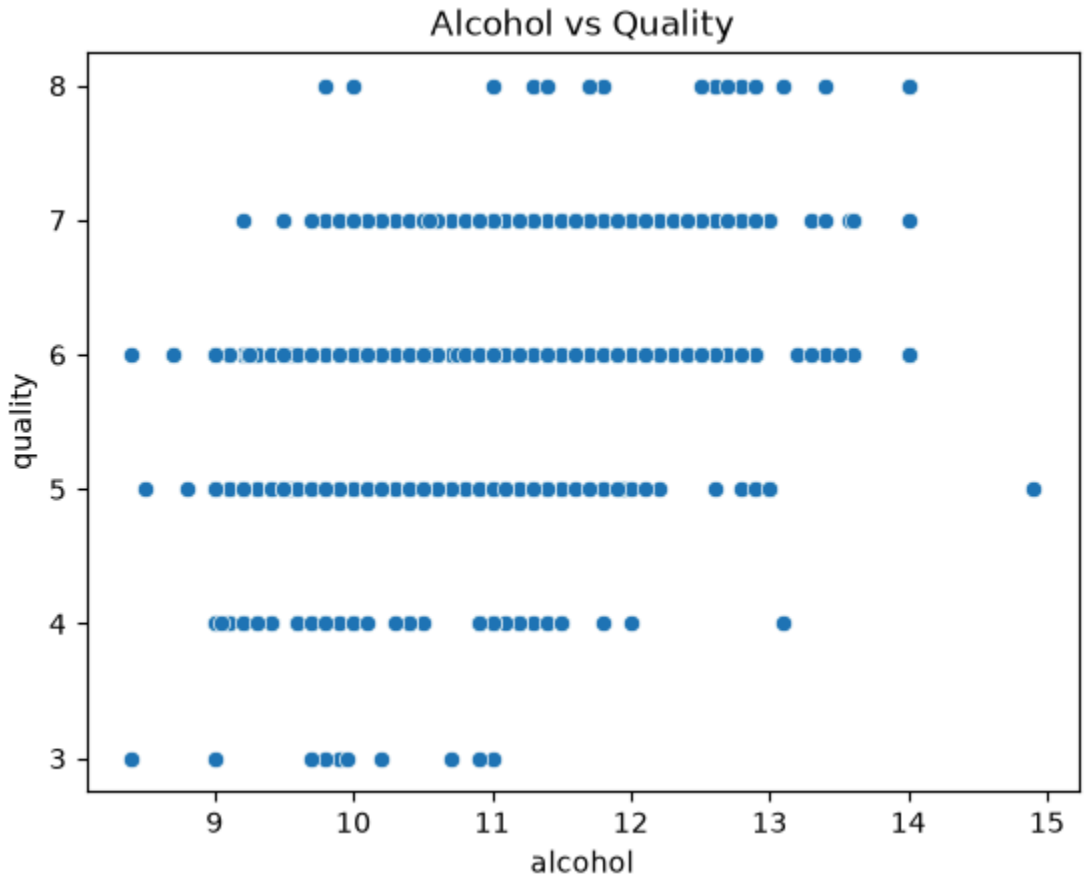

pH vs Wine QUality
pH
4.0
3.8
3.6
3.4
3.2
3.0
2.8
3
4
5
6
7
8
quality


sulphates vs Wine QUality
sulphates
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
3
4
5
6
7
8
quality

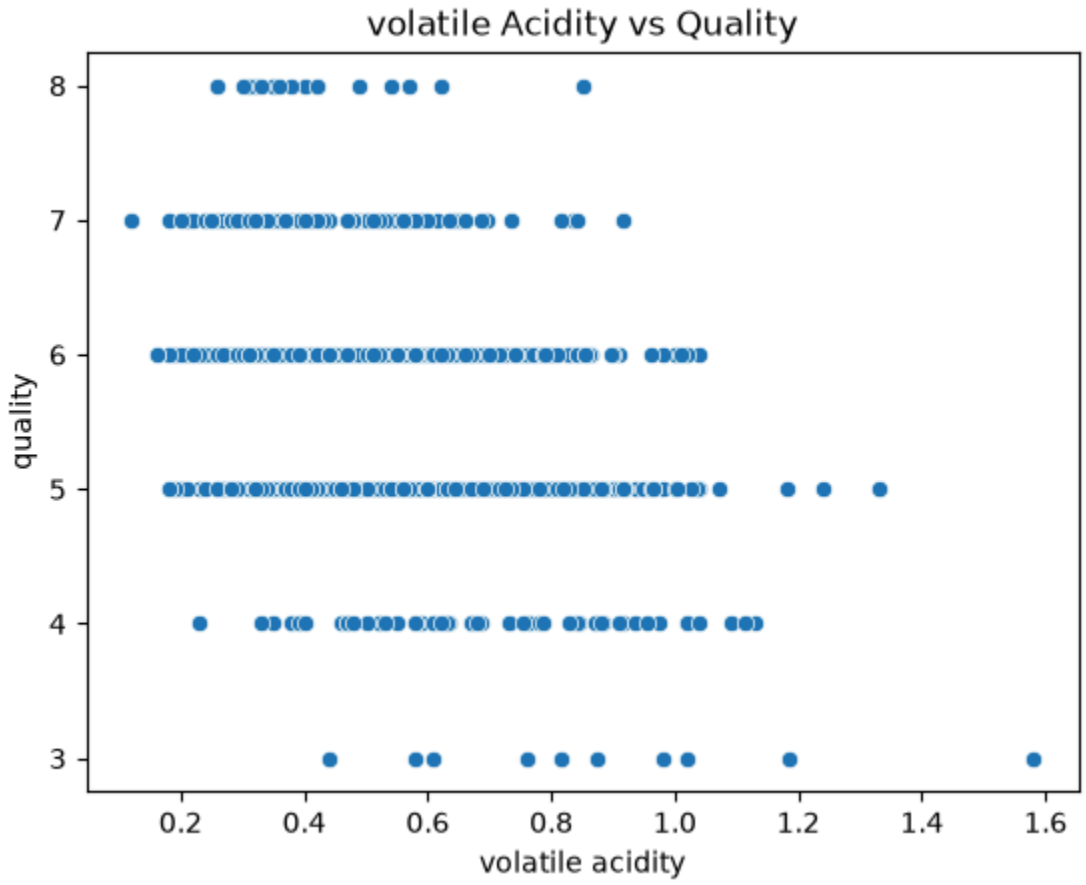




In [ ]:
```
#Scatterplot
sns.scatterplot(x='alcohol',y='quality',data=df)
plt.title('Alcohol vs Quality')
plt.show()
```



In [ ]:
```
#Scatterplot
sns.scatterplot(x='volatile acidity',y='quality',data=df)
plt.title('volatile Acidity vs Quality')
plt.show()
```

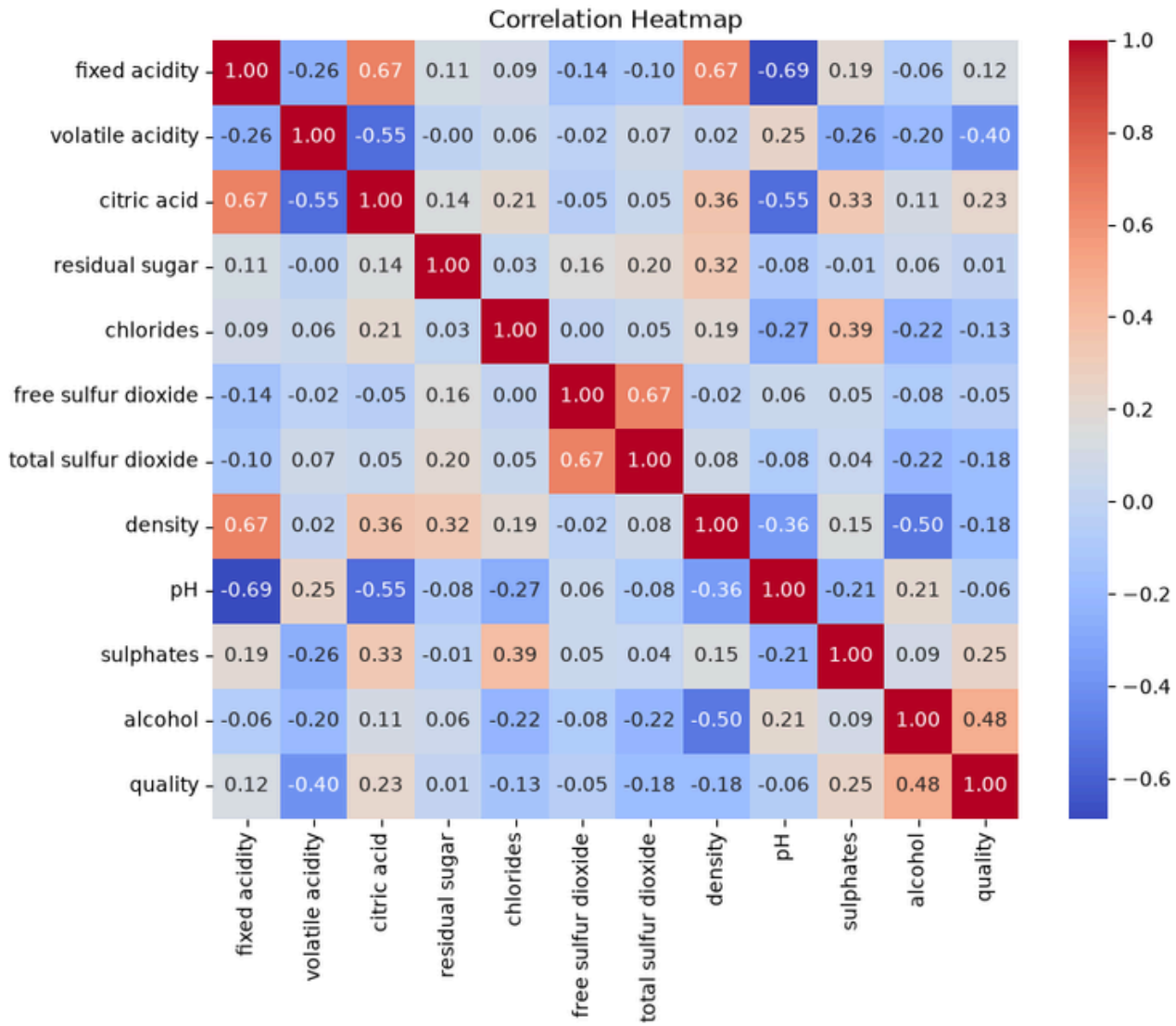


3.MultiVariate Analysis

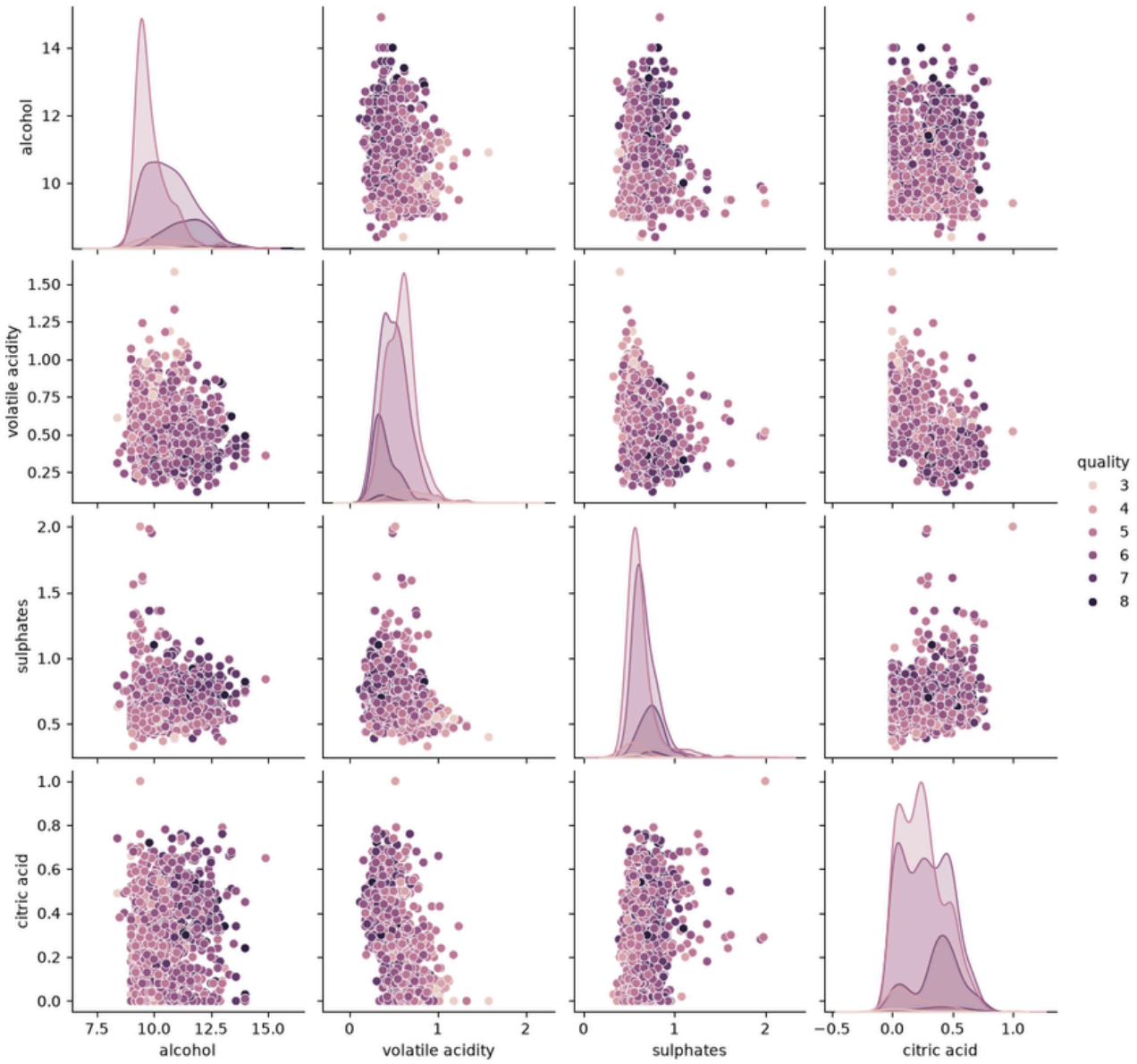
In [ ]:

```
import   matplotlib.pyplot      as  plt
import seaborn    as sns

plt.figure(figsize=(9,7))
corr=df.corr()
sns.heatmap(corr,annot=True,cmap='coolwarm',fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()
```



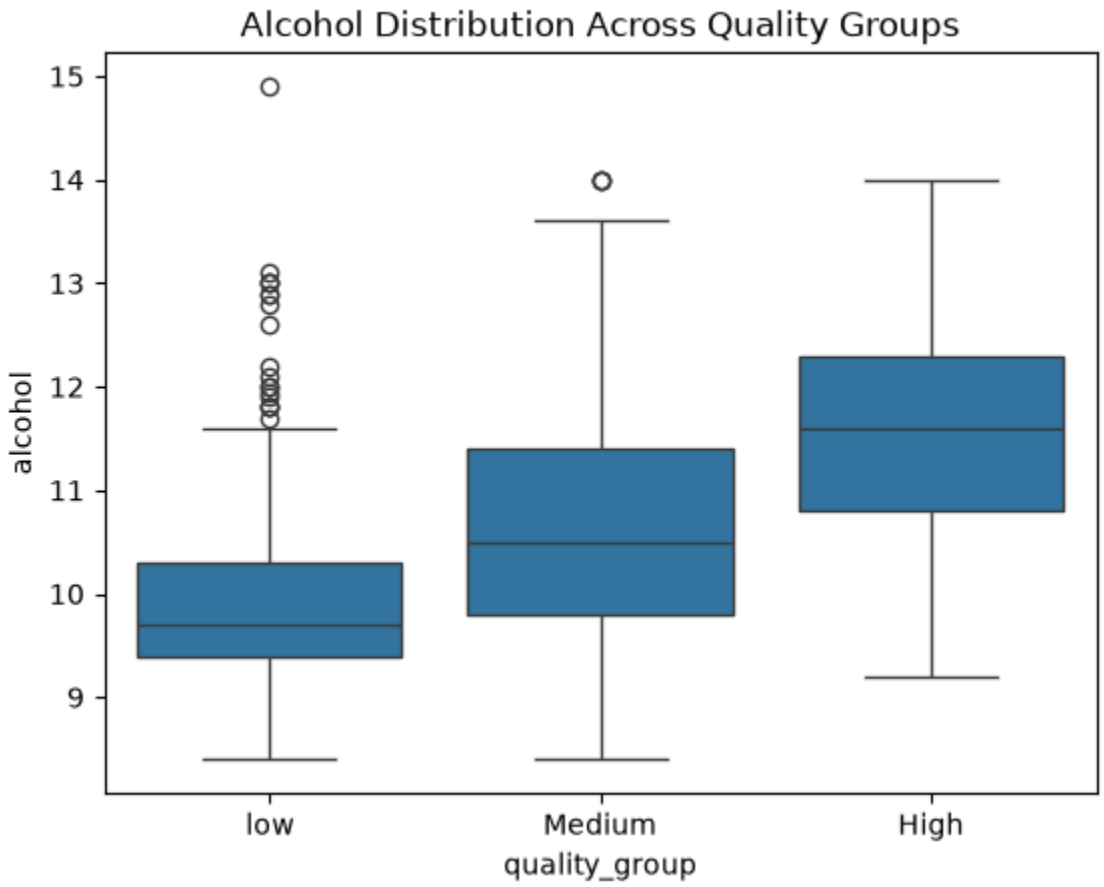
In [ ]:
```
#pair plot
selected=['alcohol','volatile acidity','sulphates','citric acid','quality
sns.pairplot(df[selected],hue='quality')
plt.show()
```



In [ ]:
```
#Multivariate Analysis using QUality Groups
df['quality_group']=pd.cut(
        df['quality'],
        bins=[0,5,6,10],
        labels=['low','Medium','High']
)
```

In [ ]:
```
sns.boxplot(

        x='quality_group',
        y='alcohol',
        data=df
)
plt.title('Alcohol Distribution Across Quality Groups')
plt.show()
```



In [ ]:
```
from  sklearn.model_selection       import   train_test_split
from sklearn.tree       import DecisionTreeClassifier,plot_tree
from sklearn.metrics       import (accuracy_score,precision_score,recall_score,
```

In [ ]:
```
X=df.drop('quality',axis=1)
y=df['quality']
```

In [ ]:
```
print("X shape",X.shape) print("y
shape",y.shape)
```

```
X shape (1359, 12)
y shape (1359,)
```

In [ ]:
```
print(y.value_counts().sort_index())
```

```
quality
3      10
4      53
5     577
6     535
7     167
8      17
Name: count, dtype: int64
```

In [ ]:
```
print(X.value_counts().sort_index())
```

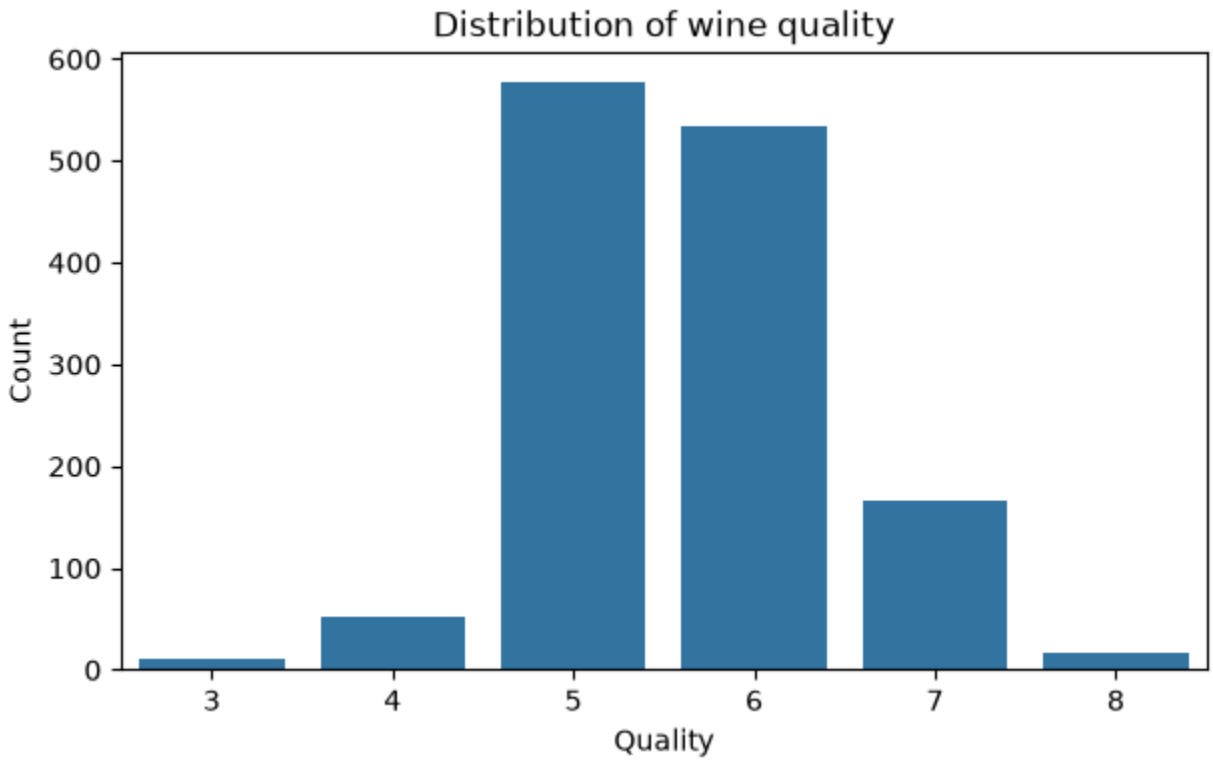
fixed acidity	volatile acidity	citric	acid	residual sugar	chlorides	free sulfur
4.6	0.520	0.15		2.1	0.054	8.0
1						
0						
0						
4.7	0.600	0.17		2.3	0.058	17.0
0						
1						
..						
15.6	0.685	0.76		3.7	0.100	6.0
0						
1						
15.9	0.360	0.65		7.5	0.096	22.0
1						
0						
0						

Name: count, Length: 4077, dtype: int64

In [ ]:

```
plt.figure(figsize=(7,4))
sns.countplot(x=y)

plt.title('Distribution of wine quality')
plt.xlabel('Quality')
plt.ylabel('Count')
plt.show()
```



In [ ]: In
```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.20,random_
```

[ ]:
```
print("X_train",X_train.shape)
print("X_test",X_test.shape)
print("y_train",y_train.shape)
print("y_test",y_test.shape)
```

```
X_train (1087, 12)
X_test (272, 12)
y_train (1087,)
y_test (272,)
```

In [ ]: In
```
criterion='entropy'
```

[ ]:
```
dt_model=DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)
```

In [ ]:
```
print(X.select_dtypes(exclude='number').columns)
```

```
Index(['quality_group'], dtype='str')
```

In [ ]: In
```
y=df['quality']
```

[ ]: In [ ]:
```
X=df.drop(columns=['quality','quality_group'],errors='ignore')
```

```
print(X.columns)
```

```
print(X.dtypes)
```

```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual suga
r',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'densi
ty',
       'pH', 'sulphates', 'alcohol'],
      dtype='str')
fixed acidity           float64
volatile acidity        float64
citric acid             float64
residual sugar          float64
chlorides               float64
free sulfur dioxide     float64
total sulfur dioxide    float64
density                 float64
pH                      float64
sulphates               float64
alcohol                 float64
dtype: object
```

In [ ]:

```
df['quality_group']=pd.cut(
    df['quality'],
    bins=[0,5,6,10],
    labels=['Low','Medium','High']
)
```

In [ ]:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X = df.drop(columns=['quality', 'quality_group'], errors='ignore')
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,

dt_model = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt_model.fit(X_train, y_train)
```

Out[ ]:

▾ DecisionTreeClassifier

Parameters

criterion	'entropy'
random_state	42
splitter	'best'
max_depth	None
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	None
max_leaf_nodes	None
min_impurity_decrease	0.0
class_weight	None
ccp_alpha	0.0
monotonic_cst	None

Fitted attributes

Name	Type	Value
classes_	ndarray[int64](6,)	[3,4,5,6,7,8]
feature_importances_	ndarray[float64](11,)	[0.06,0.13,0.08,...,0.08,0.13,0.18]
feature_names_in_	ndarray[object](11,)	['fixed acidity','volatile acidity','citric acid',...,'pH','sulphates', 'alcohol']
max_features_	int	11
n_classes_	int64	np.int64(6)
n_features_in_	int	11
n_outputs_	int	1
tree_	Tree	<sklearn.tree...001A5CC543D00>

In [ ]:
```
y_train_pred=dt_model.predict(X_train)
y_test_pred=dt_model.predict(X_test)
```

In [ ]:
```
train_accuracy=accuracy_score(y_train,y_train_pred)
test_accuracy=accuracy_score(y_test,y_test_pred)
```

In [ ]:
```
train_accuracy=accuracy_score(y_train,y_train_pred)
print(f"Training Acuuracy",train_accuracy)
```

Training Acuuracy 1.0

In [ ]:
```
test_accuracy=accuracy_score(y_test,y_test_pred)
print(f"Testing Accuracy",test_accuracy)
```

Testing Accuracy 0.4963235294117647

In [ ]:
```
print("Prediction:",precision_score(y_test,y_test_pred,average='weighted'
print("Recall:",recall_score(y_test,y_test_pred,average='weighted'))
print("F1 Score:",f1_score(y_test,y_test_pred,average='weighted'))
```

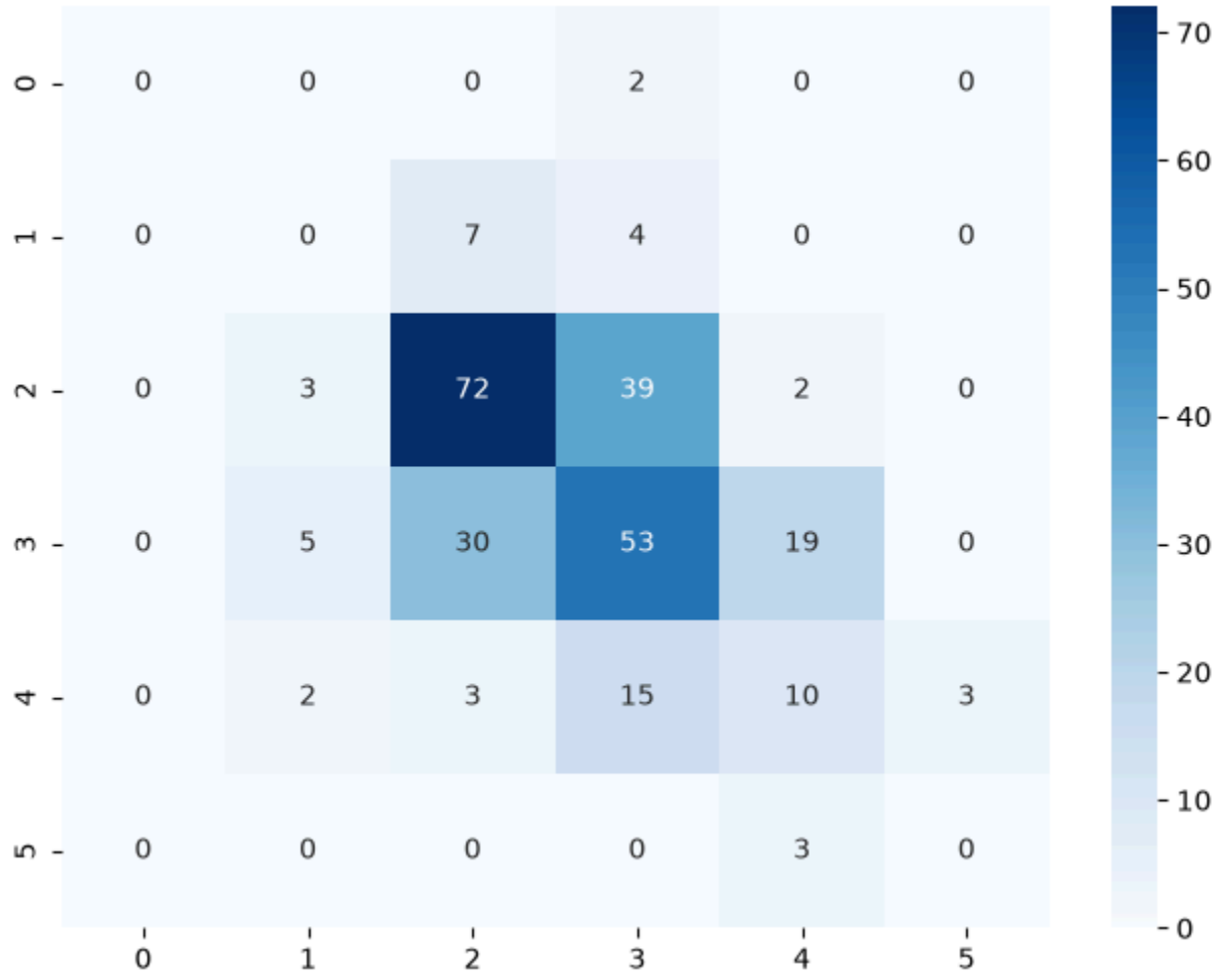
Prediction: 0.49434982217769985
Recall: 0.4963235294117647
F1 Score: 0.4951045941835992

In [ ]:
```
#confusion matrix
cm = confusion_matrix(y_test, y_test_pred)
print("Confusion Matrix:")
print(cm)
```

Confusion Matrix:
[[0 0 0 2 0 0]
[0 0 7 4 0 0]
[0 3 72 39 2 0]
[0 5 30 53 19 0]
[0 2 3 15 10 3]
[0 0 0 0 3 0]]

In [ ]:
```
plt.figure(figsize=(8,6))
sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')
```

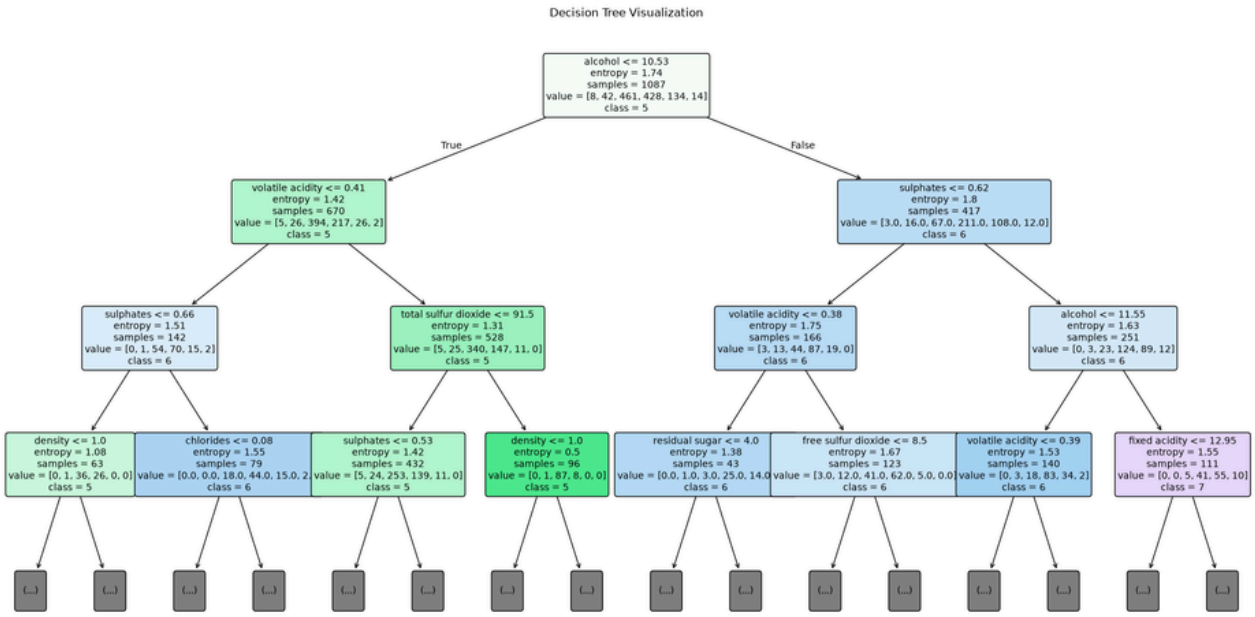
Out[ ]: <Axes: >



In [ ]:

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
plt.figure(figsize=(24,12))

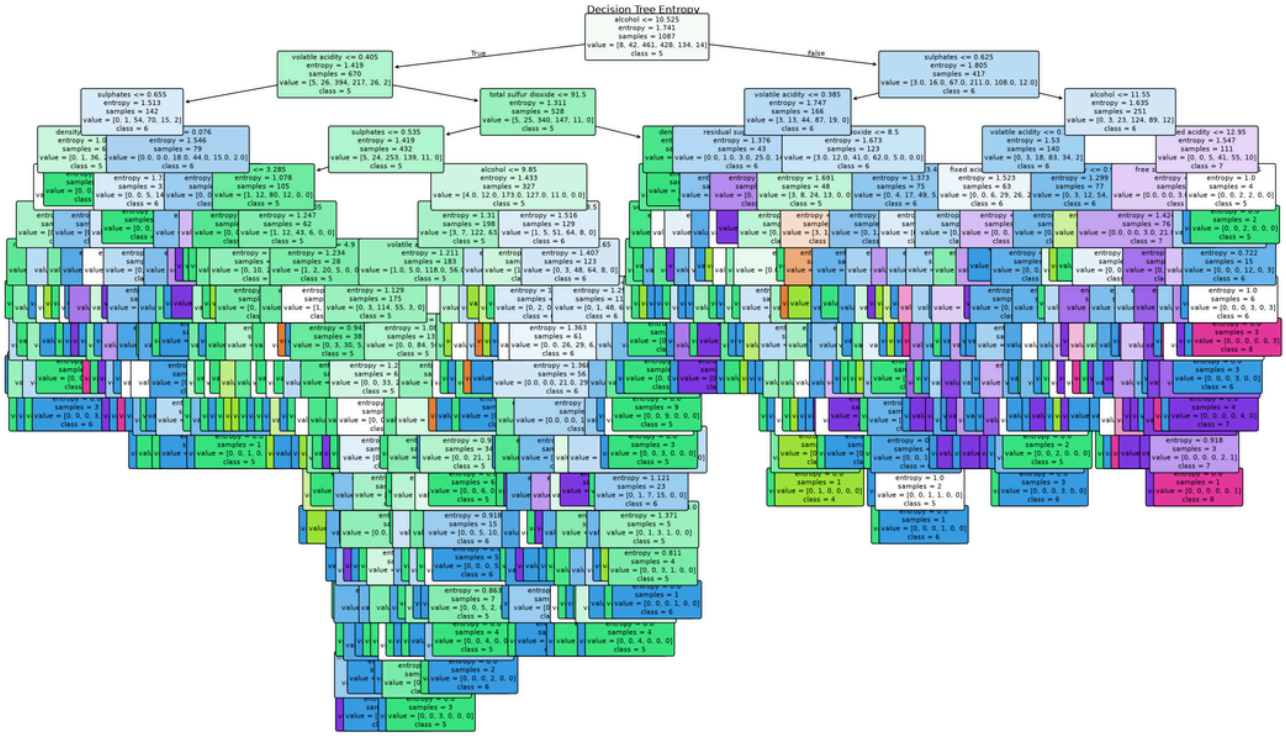
plot_tree(dt_model,feature_names=X.columns,class_names=[str(c) for c in s
plt.title(f'Decision Tree Visualization')
plt.show()
```



In [ ]:
```
print("Tree Depth",dt_model.get_depth())
print("Number of leaves:",dt_model.get_n_leaves())
```

Tree Depth 18
Number of leaves: 299

In [ ]:
```
plt.figure(figsize=(25,15))
plot_tree(
        dt_model,
        feature_names=X.columns,
        class_names=[str(c)      for c     in sorted(y.unique())],
        filled=True,
        rounded=True,
        fontsize=8
)
plt.title("Decision Tree Entropy")
plt.show()
```



AUC ROC CURVE

In [ ]:

```
from  matplotlib.typing       import   LineStyleType
from sklearn.preprocessing        import label_binarize
from sklearn.metrics       import roc_curve,auc,roc_auc_score

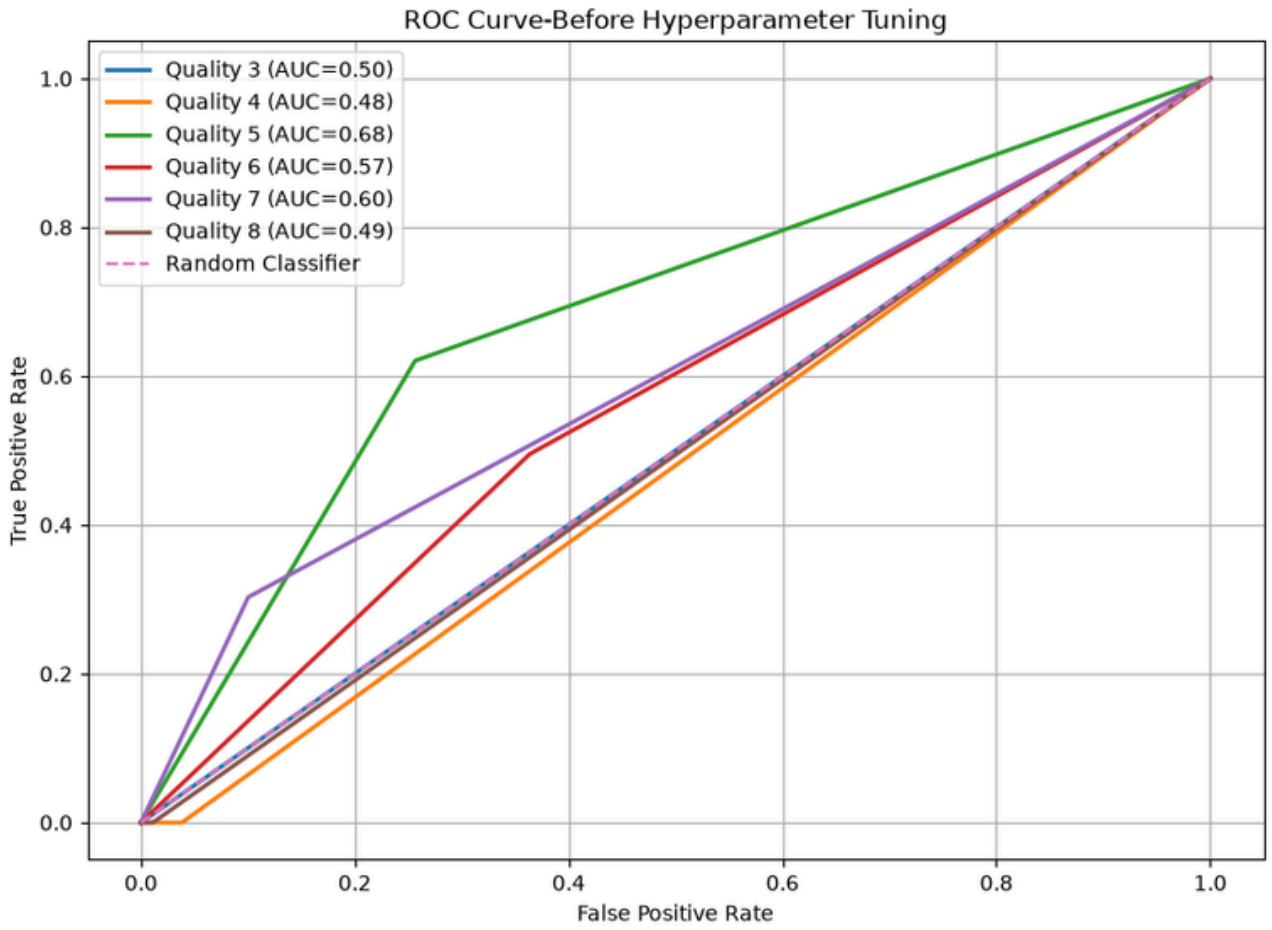
classes=sorted(y_test.unique())
y_test_bin=label_binarize(y_test,classes=classes)
y_score=dt_model.predict_proba(X_test)
plt.figure(figsize=(10,7))
for i,class_label          in enumerate(classes):
      fpr,tpr,thresholds=roc_curve(
            y_test_bin[:,i],
            y_score[:,i]
      )
      roc_auc=auc(fpr,tpr)

      plt.plot(fpr, tpr, linewidth=2, label=f'Quality {class_label} (AUC={r

plt.plot([0,       1],   [0,    1],   linestyle='--',        label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve-Before Hyperparameter Tuning')

plt.legend()
plt.grid()
plt.show()

overall_auc_before=roc_auc_score(y_test_bin,y_score,multi_class='ovr',ave
print("Overall ROC-AUC Before Tuning:",round(overall_auc_before,4))
```



Overall ROC-AUC Before Tuning: 0.615

HYPERPARAMETER TUNING

In [ ]:

```
XXXXXXXXXXXXXXXXXXXUU
```